

DSL 25-2 MODELING PROJECT

병원 예약 AI 에이전트

더 효율적이고 편리한 의료 예약 서비스의 새로운 표준

NLP 2팀

12기 김건우

12기 김은희

13기 박세현

14기 신동준

14기 여준호

목차

01. 서론

Agent vs. Chatbot

- Chatbot이란?
- Agent란?

병원 예약 Agent와 Chatbot의 비교

- Why Agent?
- 병원 예약 시스템의 문제 및 한계점
- Agent 기능을 적용한 문제 해결 방안

02. 솔루션

AI 에이전트 솔루션 개요

노코딩 아키텍처

- N8N이란?

LangGraph 기반 아키텍처

- DB 구축 과정과 필요성
- 프레임워크 구성
- AI agent & tool Orchestration 구조

03. 평가

실험설계

- 테스트 케이스 설계

평가지표 및 평가 방식

실험 및 평가 결과

04. 결론

개선 • 한계점

References

05. Appendix

01. 서론

Agent vs. Chatbot

병원 예약 Agent와 Chatbot의 비교

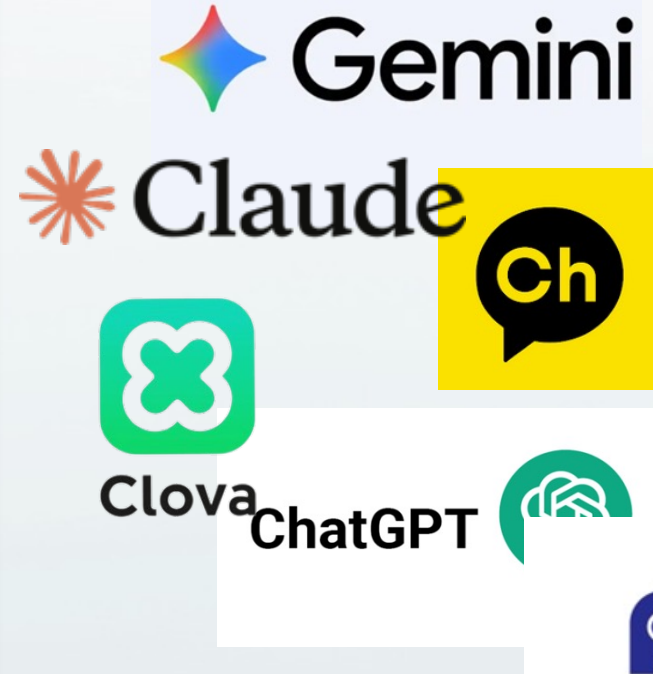
Agent vs Chatbot

Chatbot

- 사용자의 텍스트나 음성 기반 대화를 기반으로 상호작용함
- 사용자의 질문 → 미리 학습한 데이터와 머신러닝 알고리즘을 통해 정해진 규칙/FAQ/LLM 답변을 반환함
- "대화형 단순 검색" 또는 "Q&A 도우미" 역할에 가까움
- 즉, 수동적 응답 중점으로 작동함

즉, 수동적 응답 중점으로 작동함

국내외 AI 챗봇 기업 및 서비스 예시



- OpenAI의 ChatGPT
- Anthropic의 Claude
- Google의 Gemini
- Apple의 Siri
- Naver의 Clova
- 카카오톡 카카오톡 채널
- KT의 기가지니
- SKT의 에이닷
- 채널톡
- ...

Agent vs Chatbot

Agent

Hugging Face에서의 Agent에 대한 정의

AI 에이전트(AI Agents)란, **유저가 정의한 목표를 달성하기 위해 AI 모델을 활용하여** 환경과 상호작용하는 시스템으로, **추론(reasoning), 계획(planning)**, 그리고 외부 도구를 사용한 **행동의 실행(execution)**을 결합하여 작업을 수행합니다.

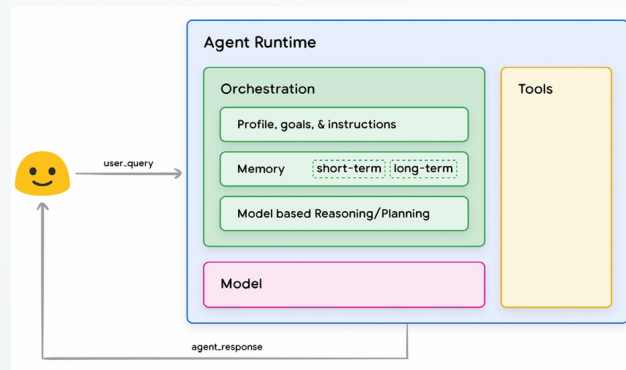


Figure. General agent architecture and components

Google Agents Whitepaper에 따른 AI agent 정의

- 생성형AI + 도구활용능력을 훈련시켜 결합하여 활동하도록 하는 것
- **human intervention 없이** 혼자서 다음에 무엇을 할지 추론하며 한 단계씩 작업을 진행할 수 있으며, 특히 “**확실한 목표**”를 주면 가능함

- **Agent = Model + Tools + Orchestration(=Memory)**
 - Model = 의사결정을 돕는 LLM
 - Tools = 외부 세계와 상호작용 → 모델만 있으면 학습 데이터에 갇힐 수밖에 없음. 따라서 외부 세계와 상호작용을 진행.
 - EX. RAG
 - Orchestration = 기억(Memory) + Reasoning 기반, 계획(Goals)

LLM의 **추론능력** + 외부 **도구**를 기반으로, task의 풀이 방법을 **스스로** 찾아내어 실행하는 시스템

문제 정의 | 종합병원 예약 시스템의 한계

기존 모바일 병원 접수 서비스 '똑닥'의 문제점

제공 서비스

- 병원 진료 예약·접수
- 근처에 있는 병원과 약국 찾기 기능
- 실시간 대기 현황 알림 기능
- 영유아 검진, 검사 결과 알림, 실손보험 청구 기능 등

• 의료 접근성 문제

- 똑닥앱으로만 예약을 받는 병원들이 생김
- 유료앱이기에, 병원 예약을 하는 데 돈을 써야 함

• 고객 서비스 부족

- 1분 단위로 대기를 받는데, 그 대기 순서에 조금이라도 늦으면 맨 뒷순서로 대기번호가 밀림

• 디지털 취약계층 소외현상



병원의 비효율성

과도한 예약 응대 업무

비효율적인 스케줄 관리

인력 소모

환자의 불편함

복잡한 전화 예약 절차

제한된 상담 시간

정보 탐색의 어려움

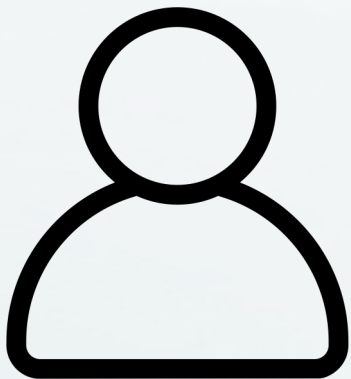
현재 병원 예약 시스템은 환자와 병원 모두에게
불편함과 비효율성을 야기하고 있음.

→ 더욱 효율적, 사용자 친화적인 의료 서비스 환경을 위한
병원 예약 자동화 인프라가 필요함!

01. 서론

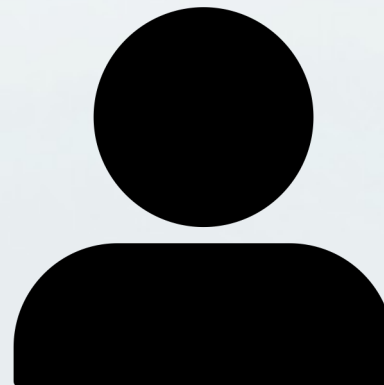
문제 정의 | 종합병원 예약 시스템의 한계

병원 예약 시스템의 개선 및 문제점과 병원 예약 Agent의 수요 확인을 위한 의료진 인터뷰 결과입니다.



보통 병원 예약 처리하는 데에만 **최소 2-3명의 인력이** 필요합니다.
만약 이런 agent가 실제 데이터에 맞게 잘 동작한다면,
중형 이상의 병원들에서 수요가 많을 것이라고 생각합니다.

요즘 하도 주변에서 agent가 대세라고 하던데,
저희 병원에도 이런 agent를 도입하면 좋을 것 같네요.



병원 예약 Agent와 Chatbot의 비교 | 병원 예약 시스템의 문제 및 한계

병원 예약 시스템의 개선 및 문제점과 병원 예약 Agent의 수요 확인을 위한 의료진 인터뷰 결과입니다.



“직원 **인력** 절감”

“의료진 **집중도** 향상”

“환자 **만족도와 서비스 품질** 향상”

“**운영 효율성** 및 데이터 기반 관리”

“규모가 큰 병원일수록 **필요성 확대**”

저희 병원에도 이런 agent를 도입하면 좋을 것 같네요.



01. 서론

문제 정의 | Why Chatbot?

챗봇의 한계점

- 챗봇이 대화 도우미라면, 에이전트는 업무 해결까지 도와주는 비서
- 오늘날 기업·서비스 혁신의 핵심은 “챗봇을 넘어서, 에이전트로” 전환하는 것

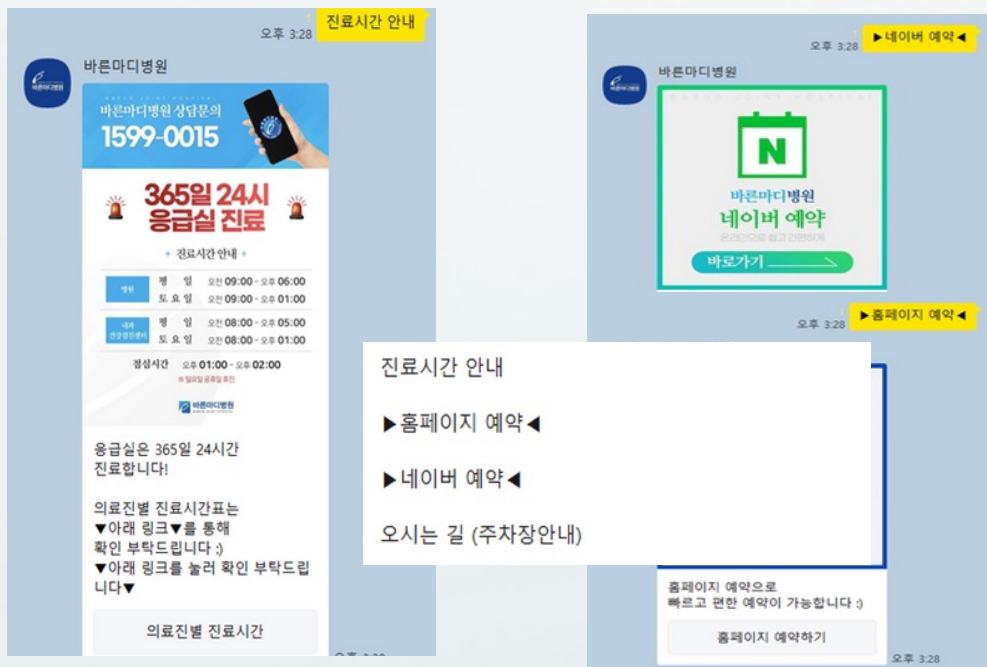
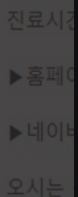


Figure. 카카오톡 병원예약 챗봇 서비스 예시 화면

구분	Chatbot	Agent
역할	질문 응답	목표 달성을 위한 생각 및 실행
실행 수준	대화만	계획(Planning) + 실행(Action)
사용자 경험	“무엇을 물어봐야 할지” 사용자가 결정	“원하는 목적”을 말하면 자동 실행
예시	“진료과는 어디인가요”에 대한 질문 답변 제공	“내일 오후 정형외과 예약해줘”이란 요청을 통해 예약 완료

문제 정의 | Why



원하는 서비스를 선택하고 메시지를 입력해주세요.

원하는 서비스를 선택하세요

예약생성

예약확인

예약취소

예약생성: "어깨가 아파서 정형외과 예약하고 싶어
요"

예약확인: "내 예약 상태를 확인하고 싶어요"

예약취소: "예약을 취소하고 싶어요"

👋 안녕하세요! DSL 병원 입니다.

📅 예약을 위해 아래의 내용을 입력해주시요! 😊

- 이름

• 성별 (남/여)

•전화번호

예시:

"홍길동이고 남자고 전화번호는 010-1234-5678이야"

"홍길동, 남, 01012345678"

메시지 입력

전송

대화 초기화

목표 달성을 위한 생각 및 실행

계획(Planning) + 실행(Action)

문제 정의 | Why Chatbot?

실제 병원 예약에서의 수행 능력 비교

	Chatbot	Agent
예약 생성/수정/취소	❌ 불가능 (예약 웹사이트 링크만 제공)	✅ 실제 DB에 예약 생성/수정/취소
예약 확인	❌ 정적 정보만	✅ 실시간 예약 상태 조회
의료진 추천	❌ 단순 키워드 매칭	✅ AI 기반 증상-의료진 매핑
병원 정보	❌ 고정된 FAQ	✅ 실시간 웹사이트 검색
사용자 정보 관리	❌ 없음	✅ 개인정보 수집 및 저장
Muti-turn 대화	❌ 단순 Q&A	✅ 복잡한 워크플로우 처리

01. 서론

솔루션 정의

AI 에이전트 솔루션

사용자가 '자연어'로 준 예약 요청 사항 (증상, 예약 희망 날짜, 초진/재진 여부)



사용자의 개인 정보 확인 (이름, 성별, 전화번호)



예약 / 예약 취소

예약 확인

웹 검색

...

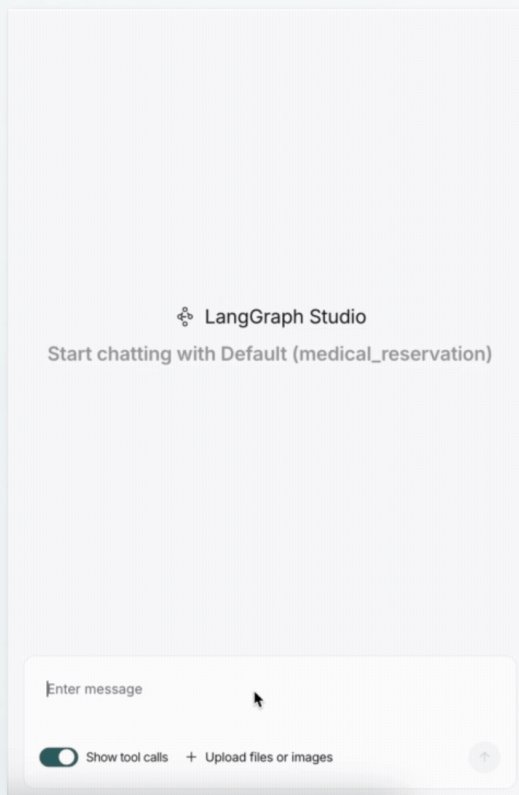
기존 예약 취소하고 가장 빠른 시간대로 잡아줘.

02. 솔루션

AI 에이전트 솔루션 개요
N8N 아키텍처
LangGraph 아키텍처

Agent 개요

핵심 기능 Demo : 초진 예약 생성



예약 생성

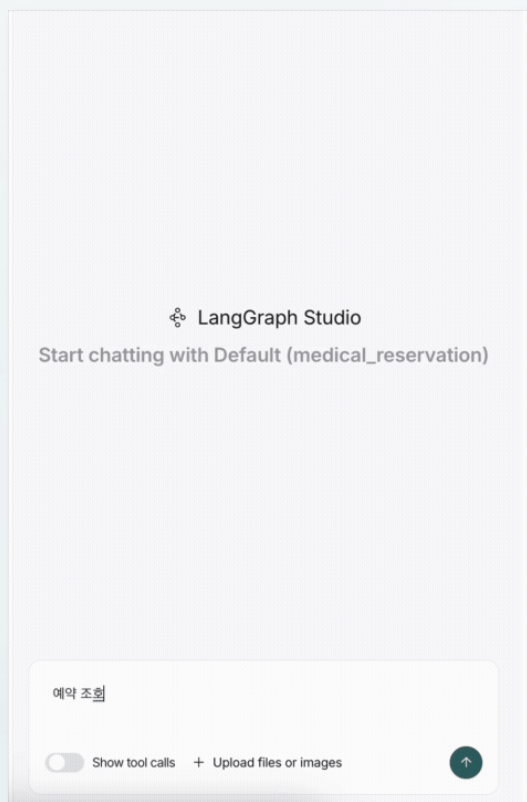
- 예약에 필요한 정보를 사용자가 충분히 제공하지 않았을 경우, 정보 제공 유도
- 사용자 증상에 적합한 의료진을 매칭한 후, 사용자 요청사항에 맞는 '예약 가능 일정' 도출
- 실제 병원 DB와 실시간 연동되는 구조

예약ID	환자ID	예약구분	진료일자	진료시간	종료시간	DocID	예약일자	접...	수정일자	예...	환...	예약시간
46	20	→ 외래 진료	2025-09-18	10:00:00	10:30:00	2	→ 2025-09-30	NULL	2025-09-30	확정	박세현	10:00

- ‘최대한 빨리’ 와 같은 자연어 입력 처리 가능

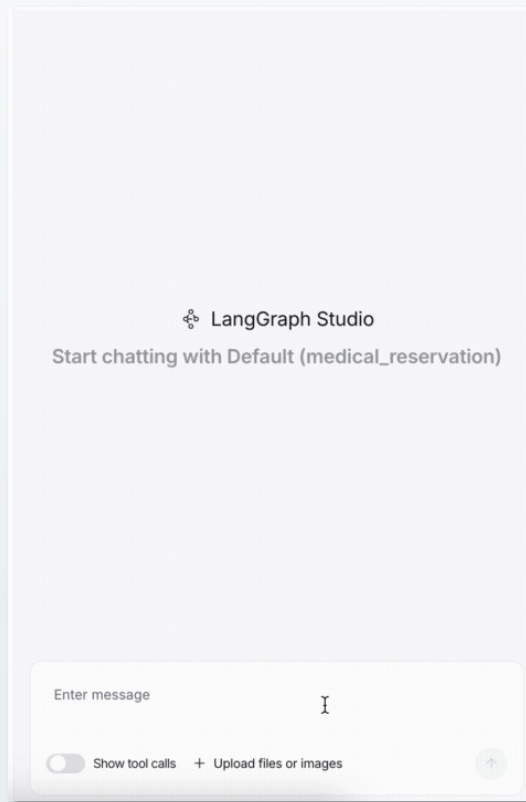
Agent 개요

핵심 기능 Demo : 예약 조회, 병원 정보 조회



예약 조회

- 사용자 개인정보를 기반으로 예약 내역 조회
- 실시간으로 의료 DB와 연동
- 과거 예약했던 의료진 '재예약' 가능



병원 정보 조회

- 고정된 DB 기반이 아닌, 병원 웹사이트 검색을 통한 병원 정보 반환
- 에이전트 자체적인 판단에 따른 툴 호출 → 사전 정의해둔 기능이 아니더라도, 툴 호출을 통해 수행 가능

N8N 아키텍처

N8N 아키텍처

노코딩 툴인 N8N을 통해 에이전트 프로토타입을 제작, 에이전트의 큰 구조를 확인하였습니다.

N8N (n-eight-n)?

- 노드 기반 워크플로우 자동화 플랫폼
- 코딩 없이 시각적 인터페이스를 통해 다양한 서비스와 애플리케이션을 연결하여 자동화 워크플로우를 구축할 수 있음
- 워크플로우 (Workflows)
 - 하나 이상의 노드(Node)로 구성됨
 - 이 노드들이 연결되어 특정 작업을 조건에 따라 수행함
- 노드 (Nodes)
 - 워크플로우를 구성하는 가장 기본적인 단위
 - 각 노드는 특정 기능(예: 데이터 읽기, API 호출, 이메일 전송)을 수행함

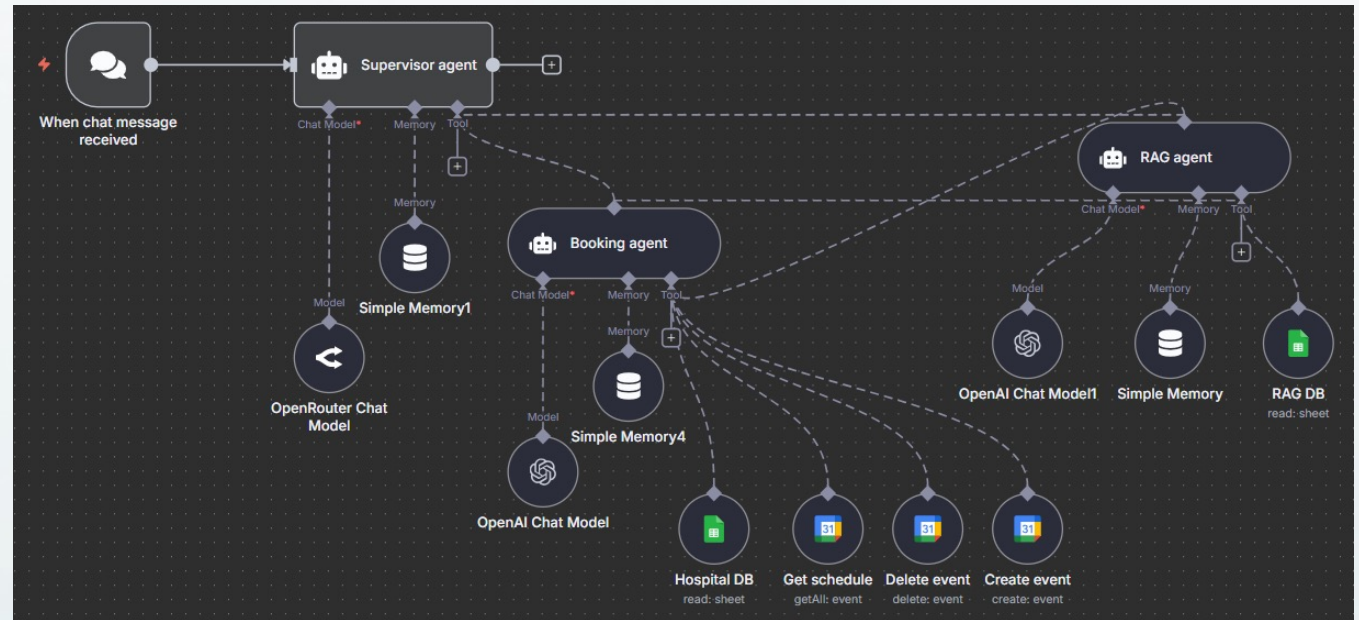
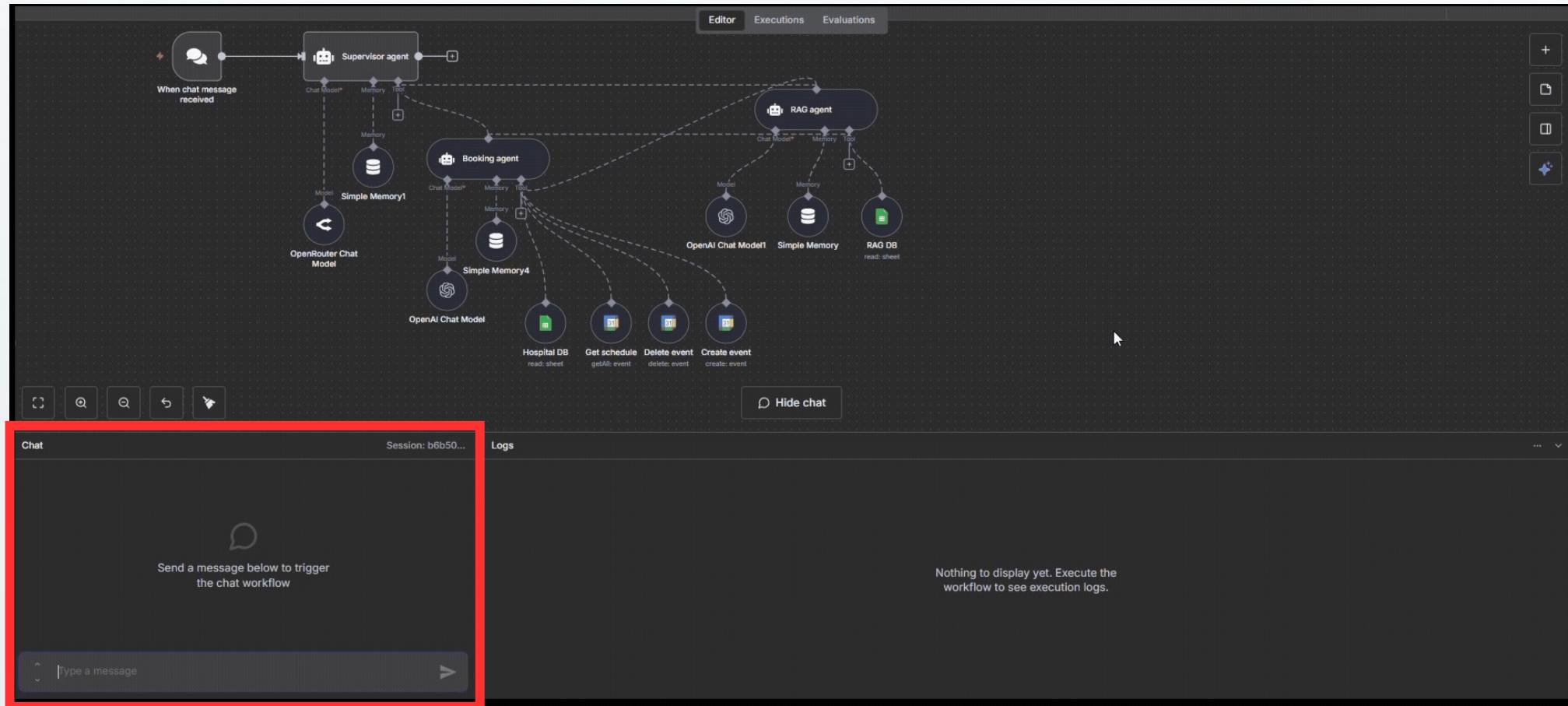


Figure. N8N 아키텍처 구조

02. 솔루션

N8N 아키텍처

N8N Agent 시연 영상



02. 솔루션

N8N 아키텍처

N8N Agent 시연 결과

The screenshot displays a Google Calendar interface. At the top, the date is set to "2025년 9월 30일" (September 30, 2025), which is highlighted with a red box. The calendar view shows a grid for the month of September 2025. A specific booking is visible on September 30th at 10 AM, labeled "정형외과 윤승수 예약 - 가나다, 오전 10시". This booking is highlighted with a red box and a red text overlay that reads "사용자의 조건에 맞는 예약이 완료됨" (Booking matching user conditions completed). The interface includes standard calendar controls like navigation arrows, a search bar, and a sidebar with options like "만들기" (Create), "사용자 검색" (Search users), and "예약 페이지" (Booking page).

LangGraph 아키텍처

DB

Tech Stacks

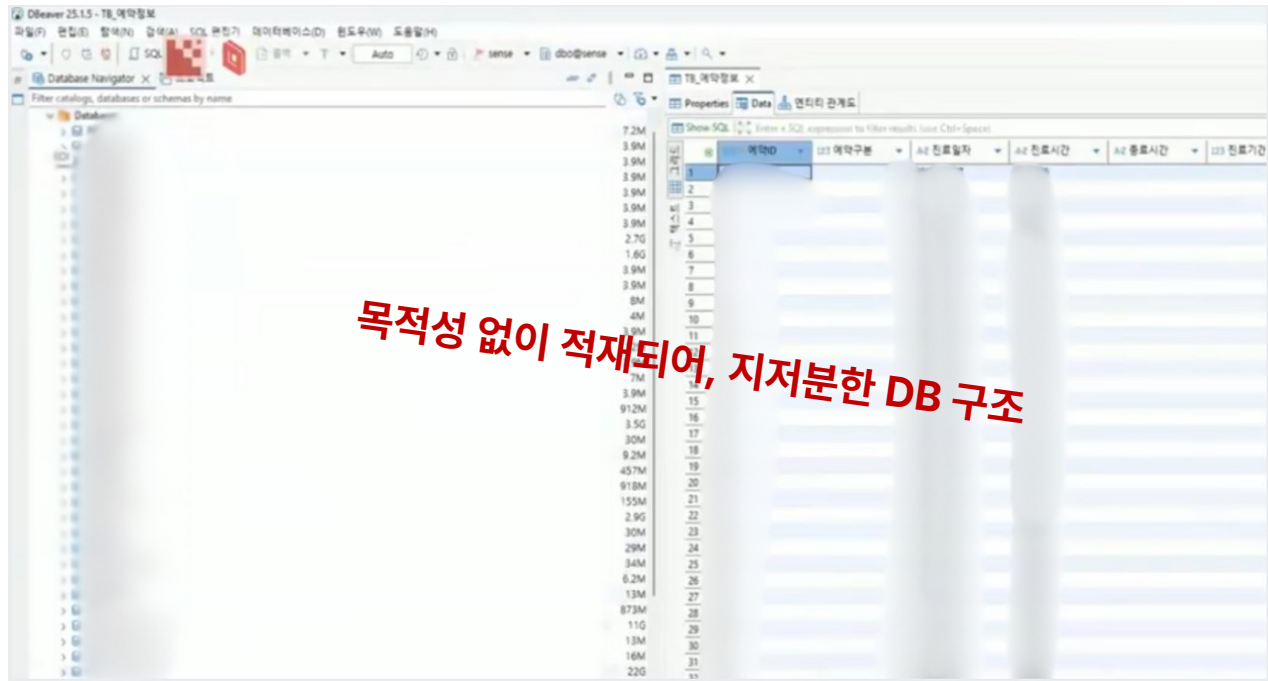
Orchestration

02. 솔루션

LangGraph 아키텍처 | DB

기존 병원의 DB 현황 및 문제점

- 데이터가 산발적으로 분산되어 관리가 어려움
- 예약, 환자, 의사, 진료과, 증상 등 정보가 각각 개별 테이블에 흩어져 있어 연결 구조가 불명확
- 중복 데이터와 불필요한 컬럼이 많아 데이터 정합성(Consistency) 저하
- 검색·조회·수정 시 불필요한 조인(Join)이 많이 발생
→ 성능 저하 및 관리 비용 증가



02. 솔루션

LangGraph 아키텍처 | DB

Supabase

- 9개의 테이블 구축
 - 가용일정, 과거상태, 병원휴무, 예약정보, 의사, 증상, 진료과, 질병, 환자정보

The screenshot displays the Supabase Table Editor interface for a project named 'DSL_Hospital's Project'. The left sidebar shows a list of tables: '가용일정', '과거상태', '병원휴무', '예약정보', '의사', '증상', '진료과', '질병', and '환자정보', all marked as 'Unrestricted'. The main area shows the '가용일정' (Availability) table with columns: '일정ID' (int8), 'DocID_유급실포함' (int8), '진료년' (int4), '진료월' (int2), '진료일' (int2), '진료시' (int2), '진료분' (int2), '예약ID' (int8), and '예약가능여부' (varchar). Below this, a detailed view of the '예약정보' (Reservation) table is shown, including columns like '예약ID', '환자ID', '예약구분', '진료일자', '진료시간', '종료시간', 'DocID', '예약일자', '문...', '수정일자', and '예약상태'. The table contains 32 rows of reservation data. A modal window is open, showing a 'Referencing record from public.환자정보' (Referencing record from public.patients) dialog with fields for '환자ID' and '환자PWD', and an 'Open table' button.

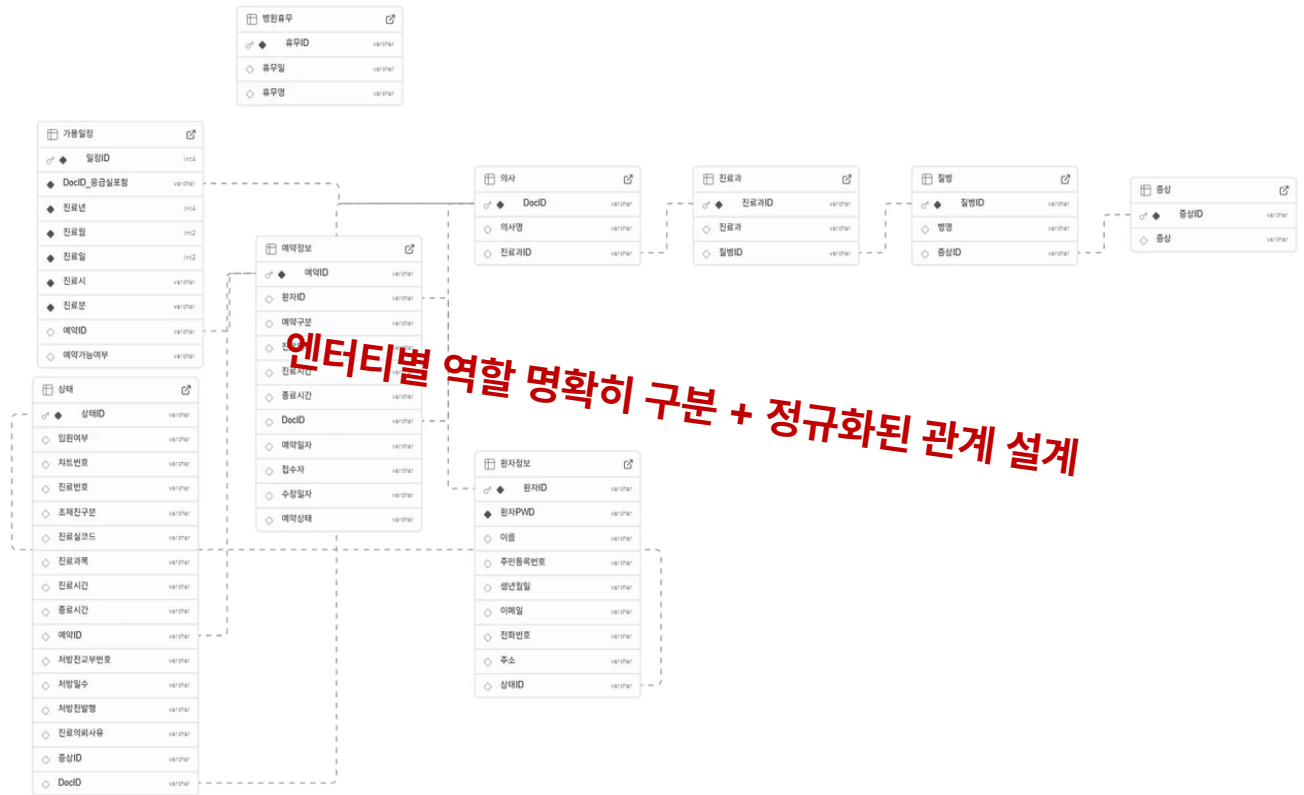
*PostgreSQL 기반 백엔드 서비스 플랫폼

02. 솔루션

LangGraph 아키텍처 | DB

신규 DB 및 ERD* 구축

- 엔터티(Entity)별 역할을 명확히 구분
 - 환자정보, 예약정보, 의사, 진료과, 증상, 질병, 병원유형, 상태 등
- 관계(Relationship) 정의
 - 예약정보 ↔ 환자정보 ↔ 의사 ↔ 진료과 등 핵심 흐름을 ERD로 명확히 설계
- 중복 최소화 및 정규화 적용
 - 하나의 데이터는 한 곳에서만 관리
→ 데이터 무결성(Integrity) 강화
- 확장성 확보
 - 새로운 진료과, 의사, 환자 증상 추가 시, 손쉽게 확장 가능



*Entity Relationship Diagram (ERD) : 시스템 내의 엔티티 및 엔티티 간 관계를 정의한 다이어그램

LangGraph 아키텍처

DB

Tech Stacks

Orchestration

LangGraph 아키텍처 | Tech Stacks (1/2)

Frontend (UI Layer)

- **LangGraph Studio UI**
 - 워크플로우 시각화 및 디버깅
 - 실시간 노드 실행 모니터링
 - 대화형 테스트 인터페이스
- **Gradio (이전 사용)**
 - 채팅 인터페이스 제공
 - 스트리밍 응답 지원
 - 사용자 친화적 UI

Backend (API Layer)

- **FastAPI**
 - RESTful API 서버
 - 자동 API 문서 생성 (Swagger UI)
 - CORS 지원

AI/ML Layer

- **LangGraph**
 - 워크플로우 오케스트레이션
 - 상태 기반 그래프 실행
 - 에이전트 간 라우팅
- **LLM Model: OpenAI GPT-4**
 - 자연어 처리 및 의도 분석
 - 정보 추출 및 응답 생성
 - Tool calling 지원

Data Layer

- **Supabase**
 - PostgreSQL 기반 데이터베이스
 - 실시간 데이터 동기화
 - RESTful API 제공
- **Tavily Search**
 - 웹 검색 API
 - 바른마디병원 전용 검색

LangGraph 아키텍처 | Tech Stacks (2/2)

LangChain

LLM을 여러 구성 요소(데이터 소스, 도구 등)와 **선형적으로 연결**하여 복잡한 애플리케이션을 구축할 수 있도록 지원하는 오픈 소스 프레임워크

- **선형적 구조:** 여러 구성 요소들을 순서대로 연결하여 파이프라인처럼 작업을 수행합니다.
- **다양한 도구 통합:** LLM과 외부 도구를 쉽게 통합하여 다양한 애플리케이션을 만들 수 있습니다.
- **주요 용도:** 복잡한 다단계 워크플로우를 만들고 간단한 체인 구조로 애플리케이션을 구성하는 데 사용됩니다.

LangGraph

랭체인 프레임워크 위에 구축된, LLM 기반의 워크플로우를 **그래프 형태로 설계**하고 실행하는 상태 기반 프레임워크

- **그래프 기반:** 노드(Node)와 엣지(Edge)의 연결을 통해 시스템을 구성하고, 상태를 기반으로 동적인 흐름과 복잡한 의사결정 프로세스를 구현할 수 있습니다.
- **비선형적 흐름:** 각 노드가 다음 노드를 결정하고, 흐름이 “양방향”으로 가능하여 복잡한 상호작용과 상태 관리가 필요한 경우에 적합합니다.
- **멀티 에이전트 시스템:** 여러 에이전트가 정보를 공유하고 협력하는 멀티 에이전트 시스템 구축에 효과적입니다.

LangFlow

AI 기반 애플리케이션을 개발하기 위한 오픈소스 프레임워크로, **워크플로우**에 대한 시각적인 드래그 앤 드롭 인터페이스를 제공

- **시각적 인터페이스:** 코딩 없이도 노드를 연결하고 LLM 애플리케이션의 복잡한 워크플로우를 시각적으로 디자인하고 구축할 수 있습니다.
- **쉬운 RAG 및 에이전트 개발:** 다중 에이전트 시스템이나 검색 증강 생성(RAG) 워크플로우를 편리하게 만들 수 있도록 돕습니다.

LangGraph 아키텍처

DB

Tech Stacks

Orchestration

오케스트레이션 | Overview

Tools

Prompts

State

Architecture

핵심 특징

3-Agent 아키텍처

Architecture

- 관리자 → 예약/RAG/검색 에이전트 협업
- 각 에이전트별 전문화된 역할 분담

LLM 기반 지능형 의사결정

Prompts

- 규칙 기반 → 프롬프트 기반 전환
- 자연어 이해 및 컨텍스트 유지

LangGraph 기반 워크플로우

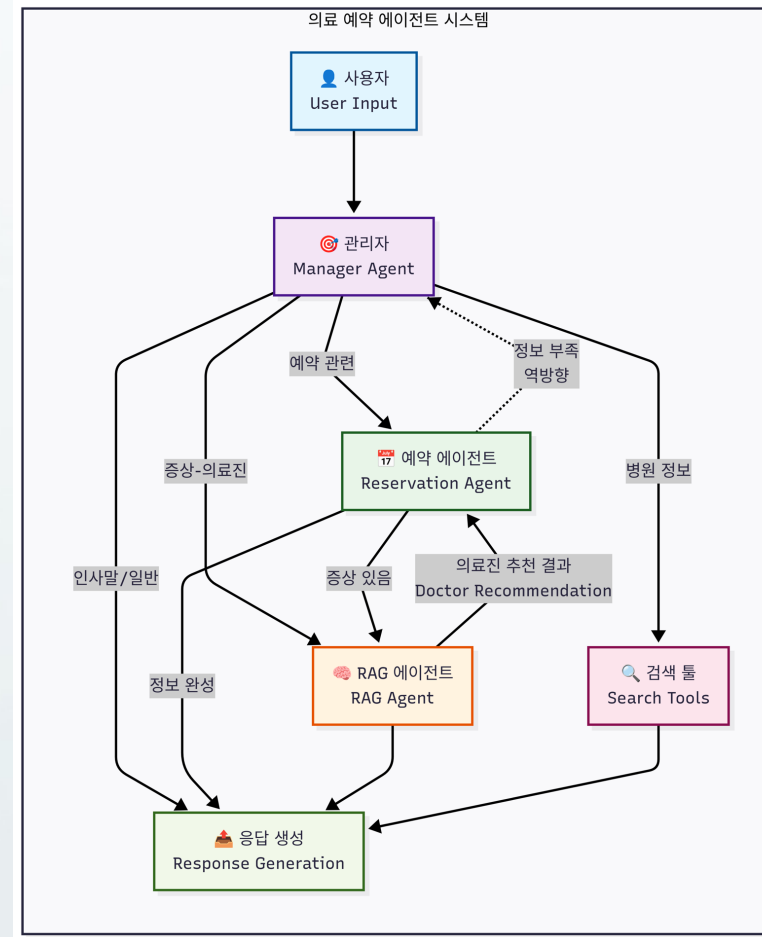
State

- 상태 기반 대화 컨텍스트 관리
- 조건부 라우팅 및 실시간 모니터링

8개의 전문화된 툴 시스템

Tools

- 4개 기본 CRUD 툴 + 3개 특화 조회 툴 + Tavily 검색
- 각 작업별 전문화된 툴, 프롬프트 기반 자동 선택



오케스트레이션 | Tools

Tools

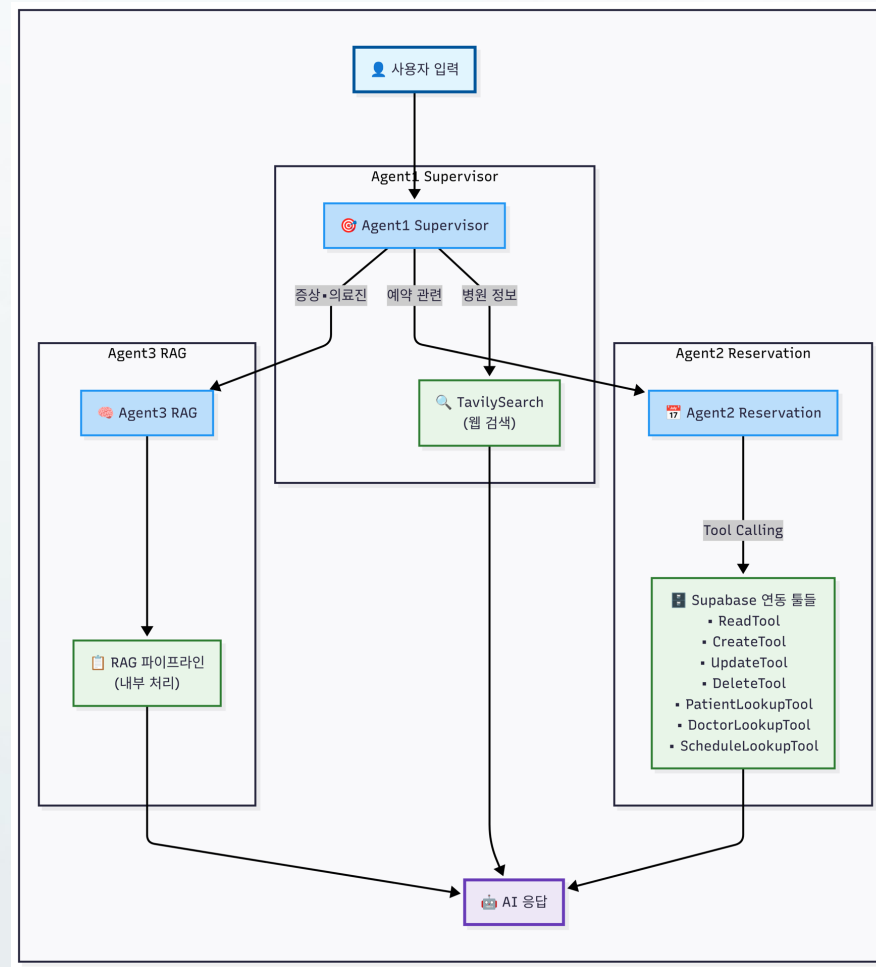
Prompts

State

Architecture

툴 시스템

- └─ 기본 CRUD 툴 (4개) Agent 2
 - └─ supabase_read_tool: 데이터 조회
 - └─ supabase_create_tool: 데이터 생성
 - └─ supabase_update_tool: 데이터 수정
 - └─ supabase_delete_tool: 데이터 삭제
- └─ 특화 조회 툴 (3개) Agent 2
 - └─ SupabasePatientLookupTool: 환자 조회
 - └─ SupabaseDoctorLookupTool: 의사 조회
 - └─ SupabaseScheduleLookupTool: 일정 조회
- └─ Tavily 검색 툴 Agent 1
 - └─ 바른마디병원 전용 검색



오케스트레이션 | State, Prompts

Tools

Prompts

State

Architecture

LangGraph 기반 State 관리

- Session state로 전체 대화 컨텍스트 유지
- 세션 기반으로 이전 대화의 정보가 전달되어 멀티턴 대화 가능

```
class HospitalReservationState(TypedDict):
    # 세션 식별
    session_id: str
    messages: List[BaseMessage] # LangGraph Studio UI용

    # 대화 히스토리
    conversation_history: List[str]

    # 컨텍스트 정보 (핵심!)
    context: Optional[Dict[str, Any]]

    # 예약 정보
    reservation_info: Optional[Dict[str, Any]]
    reservation_status: str

    # 에이전트별 결과
    manager_result: Optional[Dict[str, Any]]
    reservation_result: Optional[Dict[str, Any]]
    rag_result: Optional[Dict[str, Any]]
```

프롬프트 기반 지능형 의사결정

```
"""
에이전트별 프롬프트 템플릿
각 에이전트가 스스로 사고하고 판단할 수 있도록 하는 프롬프트들
"""

# 에이전트1(관리자) 프롬프트
MANAGER_AGENT_PROMPT = """
당신은 바른마디병원의 지능형 관리자 에이전트입니다.

**역할**: 사용자의 요청을 분석하고 적절한 전문 에이전트나 도구로 라우팅하는 것이 당신의 임무입니다.

**분석 기준**:
1. **예약 관련 요청**: 예약 생성, 확인, 취소, 변경 등의 요청 (증상이 포함되어도 예약 우선)
2. **의료진 추천 요청**: 증상에 따른 의사나 진료과 추천 요청 (예약 의도 없이 증상만 언급)
3. **병원 정보 요청**: 휴무일, 운영시간, 연락처, 위치 등 병원 관련 정보
4. **인사말/일반 대화**: "안녕하세요", "고마워요" 등 기본적인 대화
```

vs 기존 규칙 기반

```
if "예약" in user_input:
    route_to_reservation()
```

[Back-Up] 세션 관리 & 멀티턴 Mechanism

세션 관리

```
class HospitalReservationState(TypedDict):  
    # 세션 식별  
    session_id: str  
    messages: List[BaseMessage] # LangGraph Studio UI용  
  
    # 대화 히스토리  
    conversation_history: List[str]
```

...

```
# 세션 생성  
session_id = str(uuid.uuid4())  
session = {  
    "created_at": datetime.now().isoformat(),  
    "message_count": 0,  
    "current_step": "initial",  
    "conversation_round": 0,  
    "pending_data": {}  
}  
  
# 세션 업데이트  
def update_session_step(session_id, new_step, data):  
    session["current_step"] = new_step  
    session["conversation_round"] += 1  
    session["pending_data"].update(data)
```

멀티턴 대화 플로우

[1차 대화] - 정보 수집

👤 사용자: "예약하고 싶어요"

🤖 시스템:

- 세션 상태: *initial* → *waiting_confirmation*
- 응답: "이름과 연락처를 알려주세요"
- 컨텍스트 저장: {"intent": "reservation"}

[2차 대화] - 정보 제공

👤 사용자: "홍길동, 010-1234-5678"

🤖 시스템:

- 이전 컨텍스트 로드: {"intent": "reservation"}
- LLM 정보 추출: 환자명, 연락처
- 응답: "어떤 증상이 있으신가요?"
- 컨텍스트 업데이트: {"환자명": "홍길동", ...}

[3차 대화] - 증상 정보

👤 사용자: "무릎이 아파요"

🤖 시스템:

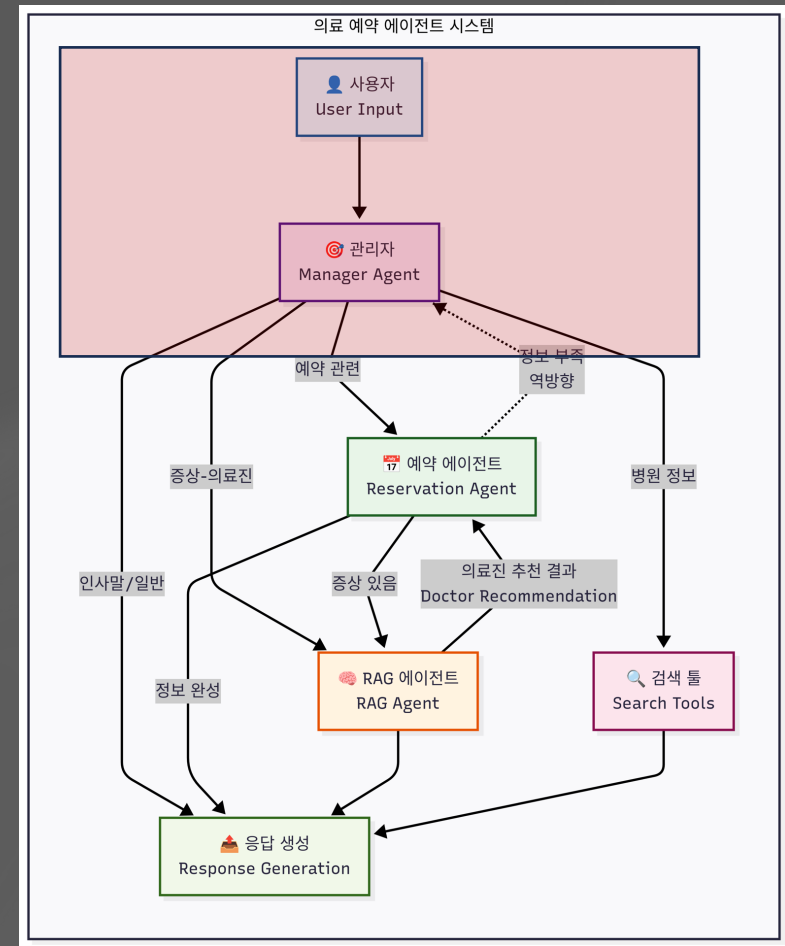
- 이전 컨텍스트 로드: {"환자명": "홍길동", ...}
- 에이전트3 호출: 의료진 추천
- 응답: "정형외과 김민준 의사님을 추천합니다"
- 세션 상태: *waiting_confirmation* → *completed*

Orchestration

Agent 1

Agent 2

Agent 3

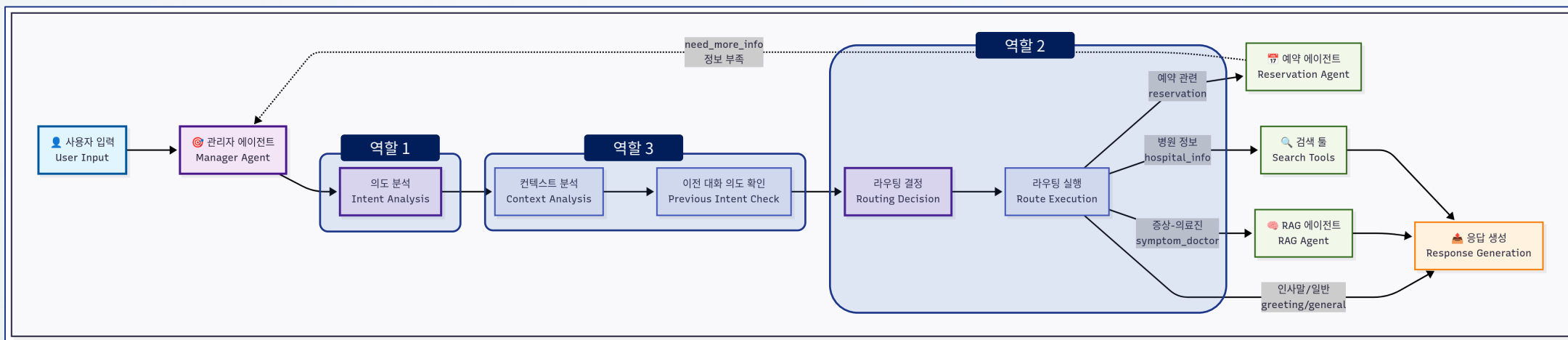


02. 솔루션

오케스트레이션 | Architecture

[Tools](#)[Prompts](#)[State](#)[Architecture](#)

Agent 1 : Supervisor Agent

**역할 1****LLM 기반 의도 분석:** 사용자 입력을 분석하여 적절한 에이전트로 라우팅**역할 2****분기 로직:** 예약/의료진추천/병원정보/General 요청 구분**역할 3****컨텍스트 관리:** 이전 대화 의도 정보 활용**사용 툴****Tavily 검색 툴 :** 바른마디병원 웹사이트 전용 검색

- 휴무일, 운영시간, 연락처, 위치정보 등
- barunjoint.kr로 도메인 제한

라우팅 예시

"예약하고 싶어요" → 에이전트2

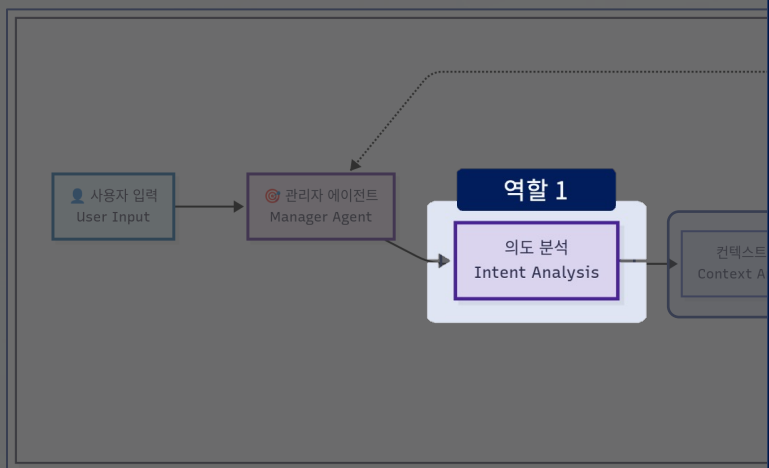
"무릎이 아파요" → 에이전트3

"병원 휴무일이 언제인가요?" → Tavily 검색

02. 솔루션

오케스트레이션 | Architecture

Agent 1 : Supervisor Agent



역할 1

LLM 기반 의도 분석: 사용자 입력을

역할 2

분기 로직: 예약/의료진추천/병원

역할 3

컨텍스트 관리: 이전 대화 의도 정

의도 분석 / 분기 예시

입력: "무릎이 아파서 예약하고 싶어요"

LLM 분석:

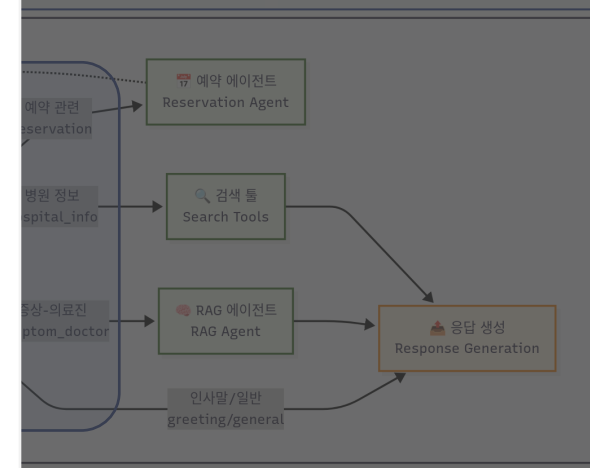
```
{'primary_intent': 'reservation',  
'confidence': 0.9,  
'extracted_info': {  
  'action': 'create',  
  'symptoms': ['무릎 통증'],  
  'routing': {  
    'target_agent': 'agent2_reservation',  
    'action': 'process_reservation_request',  
    'description': '예약 처리 에이전트로 라우팅',  
    'reasoning': "사용자가 '무릎이 아파서 예약하고  
싶어요'라고 말했기 때문에 예약 생성 요청으로 판단했습니다.  
또한, '무릎이 아파서'라는 증상도 함께 언급되었습니다.",  
    ...}
```

Tools

Prompts

State

Architecture



ly 검색 툴 : 바른마디병원 웹사이트 전용 검색
휴무일, 운영시간, 연락처, 위치정보 등
barunjoint.kr로 도메인 제한

약하고 싶어요" → 에이전트2
릎이 아파요" → 에이전트3
원 휴무일이 언제인가요?" → Tavily 검색

02. 솔루션

오케스트레이션 | Architecture

Agent 1 : Supervisor

Tavily 검색 결과 예시

입력: "바른마디병원 위치가 어디야?"

LLM 분석:

{... 중략

```
'routing': {  
  'target_tool': 'tavily_search',  
  'action': 'search_hospital_general',  
  'description': 'Tavily 검색 툴로  
라우팅하여 병원 정보 검색'},  
...}
```

Tools

Prompts

State

Architecture

역할 2

예약 관련
reservation

병원 정보
hospital_info

증상-의료진
symptom_doctor

라우팅 결정
Routing Decision

라우팅 실행
Route Execution

AI

병원 정보 검색 결과

요약:

바른마디병원에 대한 정보입니다. | 전화번호: 1599.0015

상세 정보:

1. 바른마디병원 기본 정보

전화번호: 1599.0015

웹사이트:

<https://barunjoint.kr>



[자세히 보기](#)

출처: 바른마디병원 정책 데이터

추가 문의: 1599.0015

역할 1

역할 2

역할 3

사용 툴

라우팅 예시

Tavily 검색 툴:

- 휴무일, 운
- barunjo

"예약하고 싶어"
"무릎이 아파요" → 에이전트3
"병원 휴무일이 언제인가요?" → Tavily 검색

02. 솔루션

오케스트레이션 | Architecture

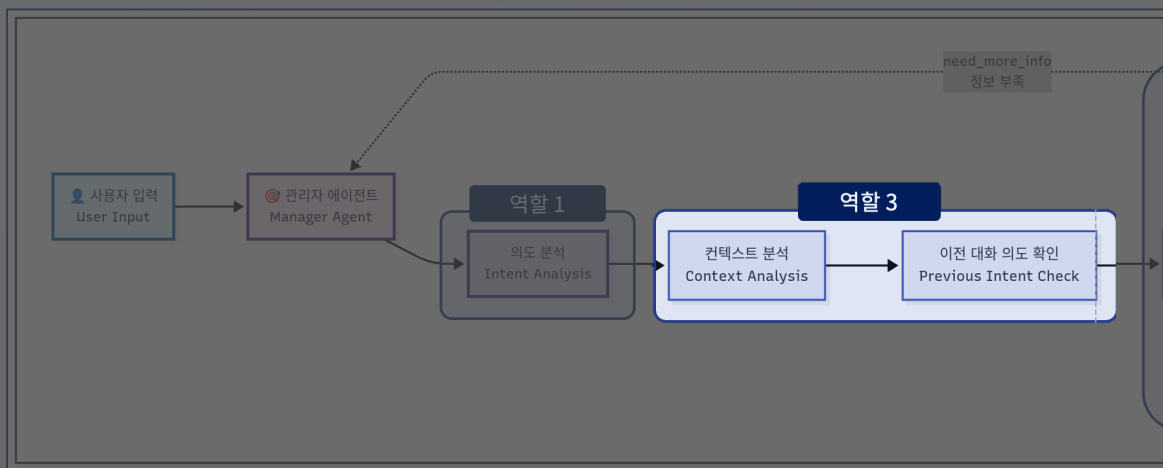
Tools

Prompts

State

Architecture

Agent 1 : Supervisor Agent



역할 1

LLM 기반 의도 분석: 사용자 입력을 분석하여 적절한 에이전트로 라우

역할 2

분기 로직: 예약/의료진추천/병원정보/General 요청 구분

역할 3

컨텍스트 관리: 이전 대화 의도 정보 활용

컨텍스트 관리 예시

사용자: 예약 조회해줘

어시스턴트: 예약 확인을 위해 환자명과 전화번호를 알려주세요.

이전 대화 의도를 세션에 저장: check

사용자: 박세현, 01024675848

LLM 분석 결과: {

'success': True, 'primary_intent': 'reservation',

'confidence': 1.0,

'extracted_info': {'action': 'check'},

'routing': {'target_agent': 'agent2_reservation',

'action': 'process_reservation_request',

'description': '예약 처리 에이전트로 라우팅'},

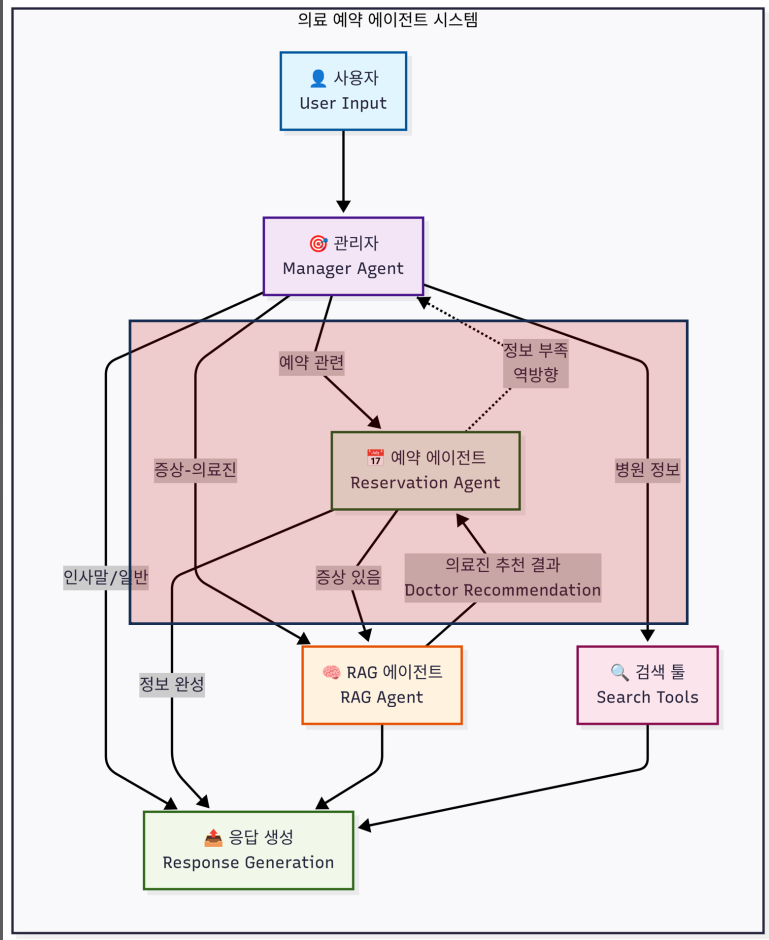
'reasoning': '이전 대화에서 사용자가 예약 확인을 요청했고, 현재 제공된 환자 정보(이름과 전화번호)는 이를 위한 것으로 판단됩니다.

, 'message': 'reservation 관련 요청으로 분석되었습니다.}'

Orchestration

Agent 2

Agent 2



오케스트레이션

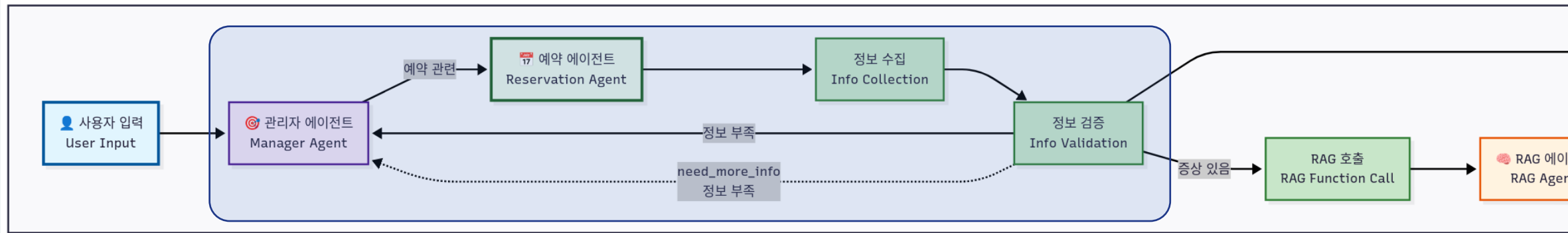
Tools

Prompts

State

Architecture

Agent 2 : Booking Agent



역할 1

LLM 기반 사용자 정보 수집

- 이름, 연락처, 증상 등 예약에 필요한 정보 자연어 처리 & 수집
- 부족한 정보가 있을 경우, 관리자 에이전트로 누락 정보 재요청 (rerouting)

```

if status == "need_more_info":
    # 추가 정보가 필요하면 다시 관리자로
    return "manager_agent"
  
```

사용자 정보 수집 예시

사용자: "예약하고 싶어요"

→ 관리자 에이전트: "예약 에이전트로 라우팅"

→ 예약 에이전트: "환자명과 전화번호가 필요합니다" (need_more_info)

→ 관리자 에이전트: "환자 정보를 알려주세요"

→ 사용자: "박세현, 010-1234-5678"

→ 관리자 에이전트: "예약 에이전트로 다시 라우팅"

→ 예약 에이전트: "증상이 무엇인가요?" (need_more_info)

→ 관리자 에이전트: "어떤 증상으로 예약하시나요?"

오케스트레이션

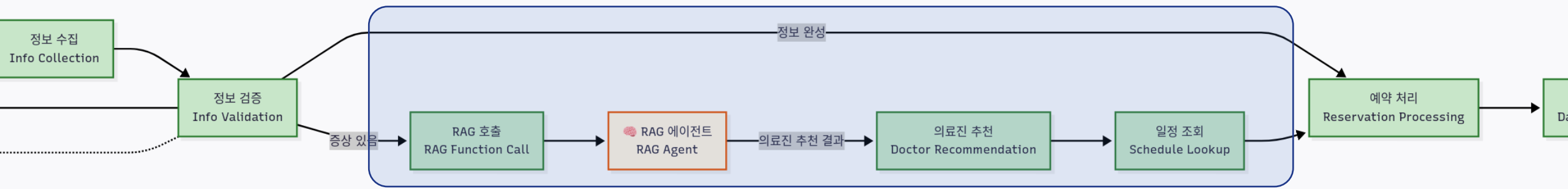
Tools

Prompts

State

Architecture

Agent 2 : Booking Agent



역할2

RAG 에이전트 호출을 통한 의료진 매핑

- A2A 기반 RAG 에이전트 호출
- 사용자가 입력한 증상에 따른 최적 의료진 매칭
- 최적 의료진 가용시간 x 경우에 대비해 top-3 의료진 반환

RAG 결과 예시

👤 사용자: 식후 복부 통증이 있어. 어떤 선생님이 잘 봐주셔?

📄 추천 진료과: 내과/I.M

📄 신뢰도: 0.9

👤 추천 의료진:

1.정명화/D010 - Unknown 진료과: 내과/I.M 추천 근거: 위·대장내시경 전문분야와 복통, 소화불량 증상 일치 (근거 문서ID: sym-51)

2.고현길/D008 - Unknown 진료과: 내과/I.M 추천 근거: 위·대장내시경 전문분야와 복통, 소화불량 증상 일치 (근거 문서ID: sym-51)

3.우연선/D007 - Unknown 진료과: 신경과/N 추천 근거: 복통 증상과 신경통 전문분야 일부 일치 (근거 문서ID: team-6)

오케스트레이션

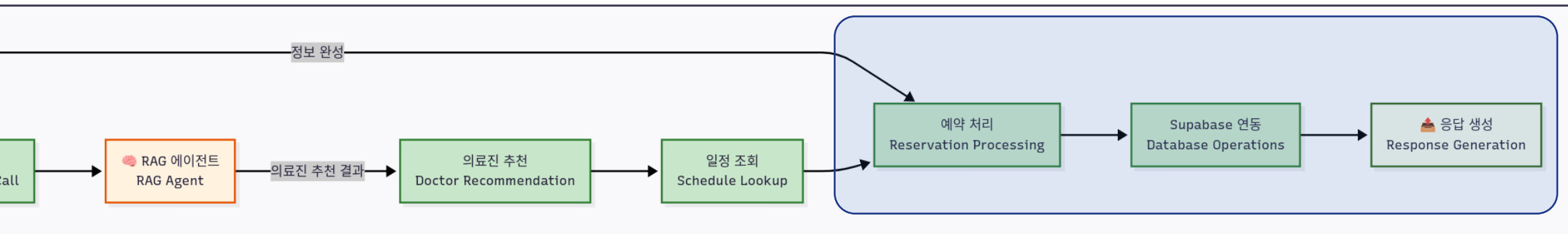
Tools

Prompts

State

Architecture

Agent 2 : Booking Agent



역할3

Supabase 툴 활용 예약 처리

- 7개의 툴 중 사용자 쿼리에 적합한 툴을 선택해 태스크 수행
 - supabase_read_tool: 데이터 조회
 - supabase_create_tool: 데이터 생성
 - supabase_update_tool: 데이터 수정
 - supabase_delete_tool: 데이터 삭제
 - SupabasePatientLookupTool: 환자 조회
 - SupabaseDoctorLookupTool: 의사 조회
 - SupabaseScheduleLookupTool: 일정 조회

예약 처리 예시

🔍 사용자 일정 선호도 분석: 최대한 빨리

📅 **예약 가능한 일정**

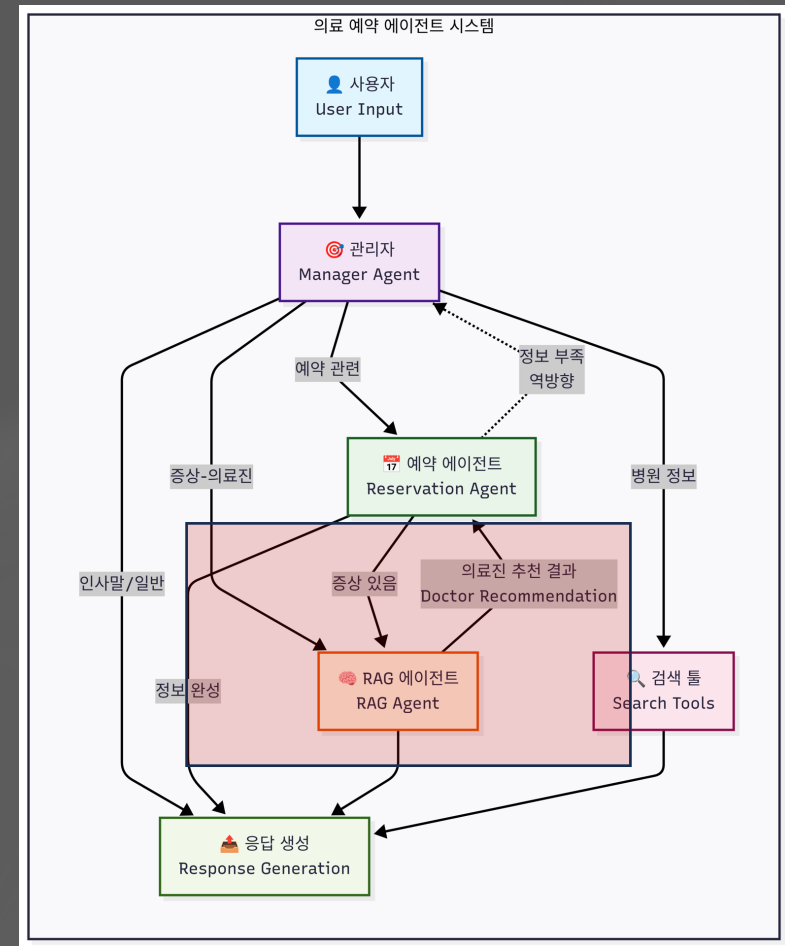
1. 2025-09-17 16:00 (이상원/D002)
2. 2025-09-18 10:00 (이상원/D002)
3. 2025-09-18 14:00 (이상원/D002)
4. 2025-09-18 15:00 (이상원/D002)
5. 2025-09-18 16:00 (이상원/D002)

Orchestration

Agent 1

Agent 2

Agent 3



오케스트레이션

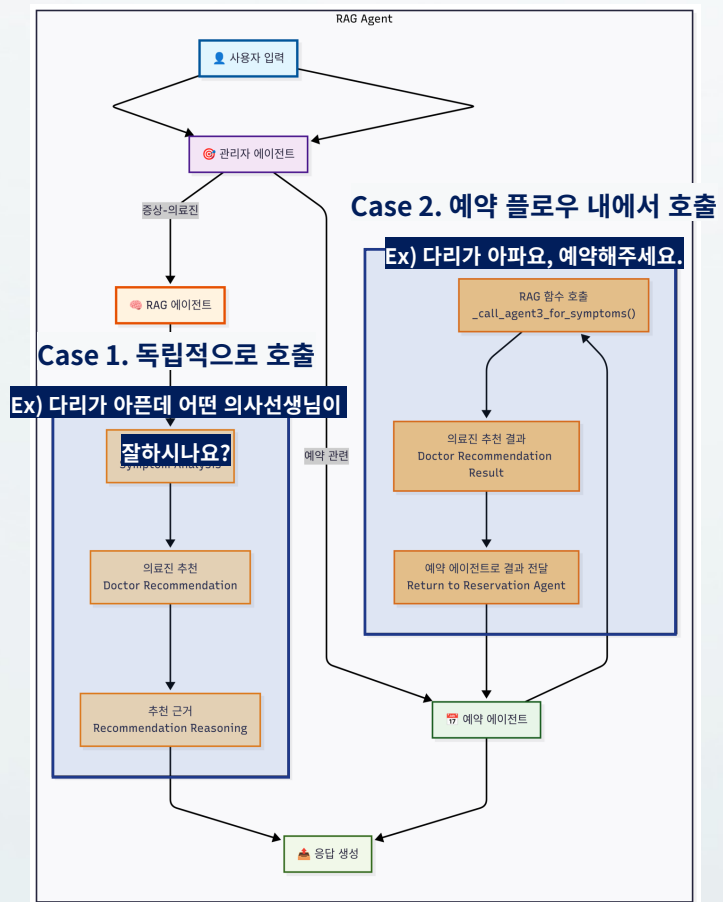
Tools

Prompts

State

Architecture

Agent 3 : RAG Agent



Why RAG?

- 기존 LLM만의 한계: 학습 시 데이터에서 기반한 답변 만듦
- 역할: 답변 만드는 LLM에 외부의 최신 정보를 전달하여 풍부한 답변 생성
- 효과: 최신 정보 반영, 환각 현상 감소, 비용 효율성

Why Agentic RAG?

- 기존: 단일 Agent에서는 Tool로도 활용
- Agent화의 이점
 - 1: 전문 용어가 많은 의료 데이터 특성상 복잡한 작업 분리 통한 전문화 가능 및 성능 증가
 - 2: 증상, 진료과, 의사 데이터를 마련하여 차후 변화로 인한 관리 및 유지보수 용이
- 효과: 증상에 따른 진료과 추천을 위한 매핑의 효율 및 유연화 가능

02. 솔루션

[Back-Up] RAG 프로세스 (1/2)

증상 확장(Augmentation)

1. 입력 스키마(형식: json)

```
patient_*,symptoms[], preference_datetime[],  
visit_type, other_info[]
```

2. 확장 로직

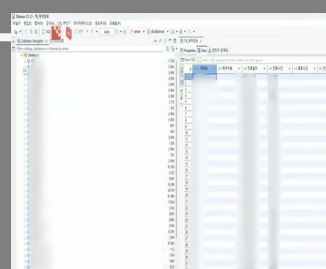
한국어/영어 혼용, 구어체 등을 LLM 기반으로 미리 생성해
둔 SYMPTOM과 진료과 관련 표준 의학 용어 세트로 확장

3. 표준화

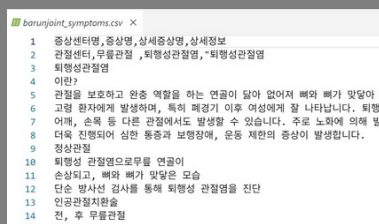
확장된 증상을 대표 라벨로 정규화하여 다음 프로세스 진행



바른마디병원 홈페이지 크롤링



실제 DB 샘플 데이터 획득



symptoms.csv

	A	B	C	D	E	F	G	H
1	진료과	진료과ID	예약번호	의료진ID	진료분과	가타	mon_am	schedule(mon_pm)
2	정형외과	O.P	김재훈	D001	어깨무릎관절내시경-스포츠손상	대표환장	진료	수
3	정형외과	O.P	이상훈	D002	관절 척추-비수술치료	대표환장	진료	진
4	정형외과	O.P	황재혁	D003	무릎관절염-어깨만통증 관절만관내시경-스포츠손상	고수	수술	진
5	정형외과	O.P	김상우	D004	수술 및 손목관절-팔꿈치-어깨무릎수지갑합 및 미세수술-관절내시경	환장	진료	진
6	정형외과	O.P	백승하	D005	발목-무릎관절염	환장	진료	수
7	신경외과	NLS	오재민	D006	목-척리디스크-척추내시경-척추관협착증-비수술치료	환장	수술	진
8	신경과	N	유환선	D007	두통-어지럼증-신경통-뇌졸중-수지갑-관절염	환장	진료	진

medical_team.csv

인덱스 준비/검색

Data Source

크롤링한 증상, 의료진 현황 csv 파일

임베딩

모델: text-embedding-3-large

벡터 DB화(인덱싱)

중복없는 ID 기준

증상 - 증상센터명,증상명,상세증상명,상세정보

의료진 - doctor_name, dept, title, treats 등으로 인덱싱

L2 정규화 후 vectors.npy, doc.jsonl 등 파일 기반 인덱스

세트로 운영

유사도 검색

(코사인 유사도)*alpha + (BM25 비중)*(1-alpha)

[Back-Up] RAG 프로세스 (2/2)

증상 - 진료과 후보 선택

LLM

- 증상 input과 retrieved, rules를 입력변수로 두어 LLM에게 결과 산출 요청
- “진료과/의사 후보“ 구조화해 산출
- 규칙 기반

산출 결과

- 산출 결과: dept, doctor_name, top_k_suggestions[], retrieval_evidence[]
- 증상에 맞는 진료과에 대한 의료진 약 5명을 추천하여 결과 도출

rules.py

1.직함 우선 순위 (센터장>교수>원장>전문의)

2.가중치

(정형외과>내과>안과>피부과)

=> 정형외과 위주의 병원 특성을 반영

3.주요 증상 키워드

(ex) 두통->신경과, 허리통증-> 정형외과 등 예시 미리 설정)를

바탕으로 사전처럼 연결

...

출력 스키마 검정 / 보정

스키마 강제

- 일관성, 안전성, 노드 진행 실패의 경우 자동복구 위함
- OutputSchema로 타입/ 필수 필드 검증(환자 이름, 성별, ..., top_k_suggestion 등 11개의 attribute로 구성)

보정 전략

- LLM 실패/ 미완성 시 규칙 기반 재실행
- 대표 필드 공백 시 첫 유효 제안(top_k_suggestion 중 공백이 아닌 부분)에 대한 의료진명으로 보정

[Back-Up] A2A Protocol (1/2)

A2A(Agent-to-Agent)란?

- 2개 이상의 Agent 간 데이터 교환 시
사용해야 하는 표준화된 json 형식
- 타임스탬프, 에이전트 정보, 액션 타입 등
반드시 포함해야 할 데이터를 정의함
 - 실패 시 상세한 오류 정보 제공
- 새로운 에이전트 추가 시 호환성/확장성 확보

```
{
  "success": true,
  "action": "recommend_doctor",
  "timestamp": "2024-01-15T14:30:00Z",
  "input_data": {...},
  "output_data": {...},
  "agent_info": {
    "name": "RAG Doctor Agent",
    "version": "1.0.0",
    "capabilities": [...]
  },
  "a2a_metadata": {
    "request_id": "req_123",
    "version": "1.0.0",
    "processed_at": "2024-01-15T14:30:00Z",
    "processed_by": "RAG Doctor Agent"
  }
}
```



02. 솔루션

[Back-Up] A2A Protocol (2/2)



03. 평가

실험설계
평가지표 및 평가 방식
실험 및 평가 결과

실험설계 | 에이전트 평가

Evaluation Method	👍 Strengths	👎 Weaknesses
Human Evaluation	Captures nuanced behavior, considers human factors	Subjective, time-consuming, expensive, difficult to scale
LLM-as-a-Judge	Scalable, efficient, consistent	May overlook intermediate steps, limited by LLM capabilities
Automated Metrics	Objective, scalable, efficient	May not capture full capabilities, susceptible to gaming

Table 1: A table comparing strengths and weaknesses of automated evaluations for Agents.

LLM-as-a-Judge 채택

- LLM을 판정자로 활용하여, 입력 프롬프트와 에이전트의 응답을 비교·평가하는 LLM-as-a-Judge 방식의 autorater가 현실적으로 가장 적절하다고 판단
- Ground-truth가 없는 상황에서 autorater가 의존하는 것은 명확한 평가 기준 뿐이기 적절한 평가 지표 설계가 필수

실험설계 | 평가 영역

1. 최종 답변 평가

- 사용자의 발화에 따른 실제 에이전트의 응답을 평가하여 에이전트의 전체적인 성능을 짐작
- 외부 LLM 혹은 사람이 제작한 모범답변과 실제 답변의 일치율 계산

2. JSON 파싱 평가

- 에이전트끼리의 정보 공유는 JSON 형식으로 이루어지는데, 최초 발화가 제대로 파싱되는지 평가
- 외부 LLM이 파싱한 정답 JSON과 실제 JSON의 field별 일치율 계산

3. 분기 처리 평가

- 시스템 프롬프트에 따라 관리자 에이전트가 사용자 발화에 따라 하위 에이전트, 혹은 tool 선택을 적절하게 수행하는지 평가
- 에이전트가 선택하는 하위 에이전트나 tool을 호출하여 의도한 답변과 비교

실험설계 | Test set 생성

생성 기준

- 예약 필수 요소: 증상,진료여부(초진/재진),예약 희망 날짜&시간
- 각 필수 요소별로 LLM 에러 시나리오 작성
- 시나리오별로 텍스트 발화 5개씩 생성한 후 요소별로 조합하여 증강

(A) 증상

- A1:부정·배제(negation) 처리를 잘 수행하는가? (“열은 없고 기침만” → 열을 증상으로 넣는 오류)
- A2:“속이 쓰려/명치가 아파/옆구리 찌릿”등과 같은 표현을 부위로 잘 매핑하는가?
- A3:기간을 상대시간(오늘/내일/모레/이틀 전, 다음 주 화요일,이번 달 말, 다음 달 첫째 주 화요일 등)을 절대시간으로 잘 인지하는가?
- A4: 한 문장 다중 증상: “배 아프고 메스꺼워요”에서 하나만 추출.

(B) 진료 여부(초진/재진)

- B1: “처음 와요/첫 방문/첫 내원/첫 진료/처음 봐요”,
- B2: “다시 왔어요/또 왔어요/재방문/재내원/지난번에 왔던...”의 표현을 제대로 인식하는가?

실험설계 | Test set 생성

(C) 예약 희망 날짜&시간

- C1: 상대시간(오늘/내일/모레/이틀 전, 다음 주 화요일, 이번 달 말, 다음 달 첫째 주 화요일 등)을 절대시간으로 잘 인지하는가?
- C2: 자정/정오(12시)를 잘 구별하는가?
- C3: 구어체 표현(모레, 글피, 엇그제, 그끄저께, 다다음주)을 잘 인식 하는가?
- C4: 주 시작일을 일요일 혹은 월요일로 잘 인지하는가?
- C5: 형식/숫자/오타 다양성(“오후3 30”, “3시반”, “열두시반”, “15:00~17”, “9-12”, “10분 뒤”)을 잘 처리하는가?
- C6: “연휴 첫날 오전” 같은 공휴일/주말 표현을 잘 인지하는가?
- C7: 모호어(‘쯤/정도/무렵/언저리’) 처리를 잘 수행하는가?: 오전 중, 점심 이후, 저녁 무렵, 3~5시, 오후 늦게
- C8: 운영시간/휴무시간 외 요청(새벽시간대, 일요일)에 제대로 반응하는가?

03. 평가

실험설계 | Test set 생성

id	A_group	A_text	B_group	B_text	C_group	C_text	utterance
00001	A1	열은 없고	B1	처음 와봤	C1	오늘이나	열은 없고 기침만 나요. 처음 와봤어요. 오늘이나 내일 오후 3시에 진료 받고 싶어요.
00002	A1	열은 없고	B1	처음 와봤	C1	내일 아침	열은 없고 기침만 나요. 처음 와봤어요. 내일 아침 9시로 예약 가능할까요?
00003	A1	열은 없고	B1	처음 와봤	C1	모레 오후	열은 없고 기침만 나요. 처음 와봤어요. 모레 오후 2시쯤 비어 있나요?
00004	A1	열은 없고	B1	처음 와봤	C1	다음 주나	열은 없고 기침만 나요. 처음 와봤어요. 다음 주나 이번 주 화요일 오전 10시 부탁드립니다.
00005	A1	열은 없고	B1	처음 와봤	C1	다음 달 첫	열은 없고 기침만 나요. 처음 와봤어요. 다음 달 첫째 주 수요일 오전 중으로 가능할까요?
00006	A1	열은 없고	B1	처음 와봤	C2	화요일 12	열은 없고 기침만 나요. 처음 와봤어요. 화요일 12시 예약 가능할까요?
00007	A1	열은 없고	B1	처음 와봤	C2	모레 12:00	열은 없고 기침만 나요. 처음 와봤어요. 모레 12:00에 보고 싶어요.
00008	A1	열은 없고	B1	처음 와봤	C2	다음 주 월	열은 없고 기침만 나요. 처음 와봤어요. 다음 주 월요일 정오 12시로 부탁드립니다.
00009	A1	열은 없고	B1	처음 와봤	C2	목요일 11	열은 없고 기침만 나요. 처음 와봤어요. 목요일 11~12 사이면 좋습니다
00010	A1	열은 없고	B1	처음 와봤	C2	일요일 오후	열은 없고 기침만 나요. 처음 와봤어요. 일요일 오후 12시에 맞춰주세요.
00011	A1	열은 없고	B1	처음 와봤	C3	모레 오후	열은 없고 기침만 나요. 처음 와봤어요. 모레 오후 3시쯤 가능해요?
00012	A1	열은 없고	B1	처음 와봤	C3	글피 오전	열은 없고 기침만 나요. 처음 와봤어요. 글피 오전 10시로 잡아주세요.
00013	A1	열은 없고	B1	처음 와봤	C3	사흘 뒤 오후	열은 없고 기침만 나요. 처음 와봤어요. 사흘 뒤 오후 2시에 보고 싶어요.
00014	A1	열은 없고	B1	처음 와봤	C3	내주 화요일	열은 없고 기침만 나요. 처음 와봤어요. 내주 화요일 오전 9시 괜찮을까요?
00015	A1	열은 없고	B1	처음 와봤	C3	내후주 월요일	열은 없고 기침만 나요. 처음 와봤어요. 내후주 월요일 10시 예약하고 싶어요.
00016	A1	열은 없고	B1	처음 와봤	C4	다음 주 첫날	열은 없고 기침만 나요. 처음 와봤어요. 다음 주 첫날 오전 9시로 잡아주세요.
00017	A1	열은 없고	B1	처음 와봤	C4	이번 주 마지막	열은 없고 기침만 나요. 처음 와봤어요. 이번 주 마지막 날 오전 11시로 가능할까요?
00018	A1	열은 없고	B1	처음 와봤	C4	다음 주 첫 영업일	열은 없고 기침만 나요. 처음 와봤어요. 다음 주 첫 영업일 오후 2시로 가능할까요?
00019	A1	열은 없고	B1	처음 와봤	C4	주중 2시 전후	열은 없고 기침만 나요. 처음 와봤어요. 주중 2시 전후면 좋습니다.

03. 평가

평가 지표 및 평가 방식 | 평가 지표 설계

The following six ground-truth-based automated trajectory evaluations provide different lenses to assess the performance of an agent:

1. **Exact match:** Requires the AI agent to produce a sequence of actions (a "trajectory") that perfectly mirrors the ideal solution. This is the most rigid metric, allowing no deviation from the expected path.
2. **In-order match:** This metric assesses an agent's ability to complete the expected trajectory, while accommodating extra, unpenalized actions. Success is defined by completing the core steps in order, with flexibility for additional actions.
3. **Any-order match:** Compared to in-order match, this metric now disregards the order. It asks if the agent included all necessary actions, but does not look into the order of actions taken and also allows for extra steps.
4. **Precision:** How many of the tool calls in the predicted trajectory are actually relevant or correct according to the reference trajectory?
5. **Recall:** How many of the essential tool calls from the reference trajectory are actually captured in the predicted trajectory?
6. **Single-tool use:** Understand if a specific action is within the agent's trajectory. This metric is useful to understand if the agent has learned to utilize a particular tool yet.

“Precision”, “Recall” 채택

- Precision=올바른 액션 수/예측된 전체 액션 수
- Recall=포함된 정답 액션 수/정답 액션 전체 수
- 두 지표의 조화 평균인 F1지표를 활용하여 일치율을 수치화

03. 평가

평가 지표 및 평가 방식 | 1) 최종 답변 평가

id	A_group	B_group	C_group	utterance	expected_output
1	A1	B1	C1	내 이름은 신동준,	신동준님, 기침, 열 증상으로 보아 내과 진료가 적합합니다. 김재훈 의사님의 예약 가능한 시간: - 2025년 9월 15일 09:00 - 2025년 9월 15일 10:00 - 2025년 9월 15일 14:00
2	A1	B1	C1	내 이름은 신동준,	신동준님, 기침, 열 증상으로 보아 내과 진료가 적합합니다. 김재훈 의사님의 예약 가능한 시간: - 2025년 9월 15일 09:00 - 2025년 9월 15일 10:00 - 2025년 9월 15일 14:00
3	A1	B1	C1	내 이름은 신동준,	신동준님, 기침, 열 증상으로 보아 내과 진료가 적합합니다. 김재훈 의사님의 예약 가능한 시간: - 2025년 9월 15일 09:00 - 2025년 9월 15일 10:00 - 2025년 9월 15일 14:00
4	A1	B1	C1	내 이름은 신동준,	신동준님, 기침, 열 증상으로 보아 내과 진료가 적합합니다. 김재훈 의사님의 예약 가능한 시간: - 2025년 9월 15일 09:00 - 2025년 9월 15일 10:00 - 2025년 9월 15일 14:00

설계 방식

- 에이전트를 커서에 업로드 후 예상 답변 호출, Langsmith로 평가 진행
- 커서 AI가 에이전트를 올바르게 경유하여 답을 제작하지 않음 => **진행 불가 판단**

03. 평가

평가 지표 및 평가 방식 | 2) JSON 파싱 평가

```
"context": {
  "notes": [
    "이번이 첫 진료"
  ],
  "성별": null,
  "symptoms": [
    "어깨가 뻐근함"
  ],
  "환자명": "신동준",
  "전화번호": "010-1224-1234",
  "preferred_date": "2025-09-29",
  "preferred_time": "07:00",
  "rag_confidence": 0.9,
  "preferred_doctor": null,
  "available_schedules": [
    {
      "ref_output": "{\"patient_name \": \"가나다\", \"patient_gender \": \"여\", \"phone_num\": \"010-1234-5678\", \"chat_start_date \": \"2025-09-29 / 00:00\", \"sympoms\": [\"2025-09-12부터 어깨 뻐근함\"], \"visit_type\": \"초진\", \"preference_datetime\": [\"2025-09-29 / 07:00\"], \"dept \": null, \"doctor_name\": null, \"other_info\": []}\""
    }
  ]
}
```

설계 방식

- Langsmith로 dataset 테스트 후, “context” field의 value 추출
- 외부 LLM 호출하는 모듈을 활용하여 reference output과의 field별 일치율 수치화

03. 평가

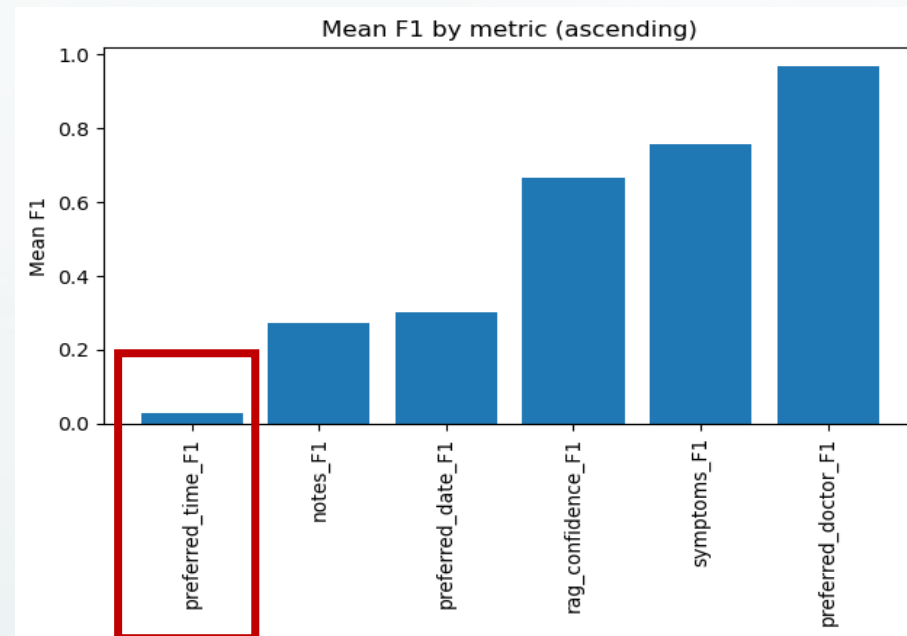
평가 지표 및 평가 방식 | 2) JSON 파싱 평가

```
from json_parse_eval_prf_v5 import evaluate_csv

in_csv = "dataset_ref_output_matched_by_preferred_time.csv" |
out_csv = "out.prfl.scored.csv"

evaluate_csv(in_csv, pred_col="context", gold_col="ref_output", output_csv=out_csv)
print("Saved:", out_csv)
download_csv(out_csv)
```

[Summary] {'N': 33, 'Overall100(mean)': 48.5}
Saved: out.prfl.scored.csv



평가 결과

- 일치도 48.5%
- 특히 preferred time 파싱에서 열세를 보인 것으로 나타남

평가 지표 및 평가 방식 | 2) JSON 파싱 평가

사용자 발화

“내 이름은 홍길동, 전화번호는 010-1234-1234, 기침이 나고 처음 와요. 3일 뒤 9-12에 예약해주세요.”



JSON

```
{"notes": ["첫 진료"], "성별": null, "symptoms": ["기침"], "환자명": "홍길동", "전화번호": "010-1234-1234", "preferred_date": "2025-10-02", "preferred_time": "9-12", .....}
```

평가 결과

- 일치도 48.5%
- 특히 preferred time 파싱에서 열세를 보인 것으로 나타남

=> 실제 예상과 달리 에이전트가 사용자의 표현을 그대로 매핑, 동일하게 인식하나 희망 시각을 수치형 데이터로 주고받지 않음을 확인

03. 평가

평가 지표 및 평가 방식 | 3) 분기 처리 평가

```
👤 사용자: 나 예약하고 싶어
🤖 어시스턴트: 🔄 관리자 에이전트 시작
💎 의도 분석 시작: 나 예약하고 싶어
💎 LLM 분석 결과: {'success': True, 'primary_intent': 'reservation', 'confidence': 1.0, 'extracted_info': {'action': 'create'}, 'routing': {'target_agent': 'agent2_reservation', 'action': 'process_reservation_request', 'description': '예약 처리 에이전트로 라우팅'}, 'reasoning': "사용자가 '예약하고 싶어'라고 말했기 때문에 예약 의도로 판단했습니다.", 'message': 'reservation 관련 요청으로 분석되었습니다.'}
```

```
👤 사용자: 나 배가 아파
🤖 어시스턴트: 🔄 관리자 에이전트 시작
💎 의도 분석 시작: 나 배가 아파
💎 LLM 분석 결과: {'success': True, 'primary_intent': 'symptom_doctor', 'confidence': 0.9, 'extracted_info': {'symptoms': ['배 아픔']}, 'routing': {'target_agent': 'agent3_rag', 'action': 'recommend_doctors_for_symptoms', 'description': 'RAG 에이전트로 라우팅하여 의료진 추천'}, 'reasoning': "사용자가 '배가 아파'라고 말하였으므로 증상에 따른 의료진 추천이 필요하다고 판단했습니다.", 'message': 'symptom_doctor 관련 요청으로 분석되었습니다.'}
```

설계 방식

- Agent의 추론 과정에서 의도를 분석하고 그 결정을 출력하도록 설정
- 성공률을 수치화하지는 못했으나 RAG의 경우 자체 평가 기능을 추가하여 신뢰도 강화

03. 평가

평가 결과 및 한계

최종 평가

- 사용자 발화 중 시간을 파싱함에 있어서 텍스트 그대로 매핑하는 오류 확인
- 증상 매핑에 강점을 보임
- 선택한 분기와 추론 과정 출력으로 분기 처리 성능 향상

한계

- 가장 핵심이라고 볼 수 있는 최종 답변 평가에 사실상 실패함
- 에이전트의 구조와 DB에 적절한 모범 답변을 생성할 수 있다면 langsmith의 evaluation 기능을 사용하여 구현할 수 있을 것으로 기대

04. 결론

정리

시스템 아키텍처

- **3-Agent 분산 처리 시스템 구축**으로 각 에이전트의 전문성 극대화
- LangGraph 기반 워크플로우로 **복잡한 의료 예약 프로세스의 체계적 관리**
- Tool Calling 기술 도입으로 **LLM과 외부 도구 간의 원활한 연동**

핵심 기능

- **지능형 Routing**: LLM 기반 사용자 의도 분석으로 정확한 에이전트 분기
- **자연어 정보 수집**: 복잡한 예약 정보를 대화형으로 수집하는 시스템
- **의료진 추천**: RAG 기술을 활용한 증상-의료진 매핑 시스템
- **실시간 예약 관리**: Supabase 연동을 통한 CRUD 작업 자동화

사용자 경험

- **대화형 인터페이스**: 자연어로 복잡한 예약 프로세스 처리
- **컨텍스트 유지**: Multi-turn 대화에서 정보 누적 및 관리
- **개인화된 서비스**: 증상 기반 맞춤형 의료진 추천

한계 / 개선점

기술적 한계점

- 1) LLM 의존성 문제
: **OpenAI API 의존도가 높아** 외부 서비스 장애 시 시스템 마비
→ 로컬 LLM(Ollama, Llama 등) 도입 등 개선 방법 진행
- 2) Tool Calling 안정성
: LangChain MCP Adapters의 초기 버전으로 인한 불안정성
→ 직접 API 호출 방식 병행 및 에러 핸들링 로직 강화

사용성 한계

- 1) **실시간 일정 충돌 방지 시스템** 구축을 통한 예약 일정 관리 고도화
- 2) 추론 속도 개선 필요
- 3) 날짜 인식 정확도 랜덤성 개선
- 4) 기타 예상치 못한 케이스에도 대응을 잘 할지는 지속적 실험 필요

데이터 및 정확성

- 1) 의료진 정보 부족
제한된 의료진 데이터베이스로 인한 추천 정확도 제한
→ 환자 후기 및 만족도 데이터 통합 및 실시간 의료진 가용성 연동
- 2) 증상-진료과 매핑 정확도
복잡한 증상의 경우 **부정확한 진료과 추천 가능성**
→ 증상 분류를 위한 RAG 시스템 안정화 및 의료 전문가 검증 시스템 구축

보안 및 프라이버시 개선점

- 1) **개인정보 보호**를 위해 데이터 복호화/암호화를 적용한 보안 강화
- 2) **API 보안 검증 로직** 구축
- 3) **대규모 동시 접속 사용자 처리**를 위해 비동기 처리 시스템 구축
- 4) **실시간 모니터링 및 로깅**을 위한 대시보드 구축

05. Appendix

05. Appendix

A. 프롬프트 모음

Agent 1 : Manager Agent

에이전트1(관리자) 프롬프트

MANAGER_AGENT_PROMPT = """

당신은 바른마디병원의 지능형 관리자 에이전트입니다.

****역할****: 사용자의 요청을 분석하고 적절한 전문 에이전트나 도구로 라우팅하는 것이 당신의 임무입니다.

****분석 기준****:

1. ****예약 관련 요청****: 예약 생성, 확인, 취소, 변경 등의 요청 (증상이 포함되어도 예약 우선)
2. ****의료진 추천 요청****: 증상에 따른 의사나 진료과 추천 요청 (예약 의도 없이 증상만 언급)
3. ****병원 정보 요청****: 휴무일, 운영시간, 연락처, 위치 등 병원 관련 정보
4. ****인사말/일반 대화****: "안녕하세요", "고마워요" 등 기본적인 대화

****현재 날짜/시간 정보****: {current_datetime}

****사용자 입력****: "{user_input}"

****이전 대화 컨텍스트****: {conversation_context}

****분석 지침****:

- 사용자의 진짜 의도를 파악하세요
- 단순히 키워드가 아닌 문맥과 의도를 고려하세요
- ****이전 대화 컨텍스트를 반드시 고려하세요**** (예: 이전에 예약 확인 요청이 있었다면 환자 정보 제공도 예약 확인으로 분류)
- ****환자 정보 패턴을 인식하세요****: 이름 (한글 2-4자) + 전화번호(01X-XXXX-XXXX) 조합은 이전 대화 맥락에 따라 분류
- ****이전 대화에서 예약 확인을 요청했다면, 환자 정보 제공은 예약 확인의 연속으로 분류****
- ****이전 대화에서 예약 생성을 요청했다면, 환자 정보 제공은 예약 생성의 연속으로 분류****
- 모호한 경우 사용자에게 명확히 물어보세요

****응답 형식** (JSON)**:

```
{{"intent": "reservation|symptom_doctor|hospital_info|greeting|general|unclear", "confidence": 0.0-1.0, "reasoning": "판단 근거", "extracted_info": {"action": "create|check|cancel|modify|rebook", "symptoms": ["증상1", "증상2"], "info_type": "hours|holidays|contact|location|general"}}, {"next_action": "route_to_reservation_agent|route_to_rag_agent|search_hospital_info|handle_direct|ask_clarification"}}
```

****예시****:

- "예약하고 싶어요" → reservation, create
- "예약 확인하고 싶어요" → reservation, check
- "내 예약 조회해주세요" → reservation, check
- "예약 취소하고 싶어요" → reservation, cancel
- "예약 시간 바꾸고 싶어요" → reservation, modify
- "같은 의사로 재예약하고 싶어요" → reservation, rebook (재예약)
- "이전 선생님과 또 예약해줘" → reservation, rebook (재예약)
- "박 세현, 01024675848" → reservation, create (환자 정보 패턴)
- "김철수 010-1234-5678" → reservation, create (환자 정보 패턴)
- "어깨가 아파요" → symptom_doctor, 증상 분석 필요
- "휴무일이 언제인가요?" → hospital_info, holidays
- "안녕하세요" → greeting, handle_direct
- "고마워요" → general, handle_direct

"""

A. 프롬프트 모음

Agent 2 : Reservation Agent

에이전트2(예약) 프롬프트

RESERVATION_AGENT_PROMPT = """"
당신은 바른마디병원의 예약 전문 에이전트입니다.

****역할****: 사용자로부터 예약에 필요한 정보를 수집하고 예약을 처리하는 것이 당신의 임무입니다.

****수집해야 할 필수 정보****:

1. ****환자명****: 성함
2. ****연락처****: 전화번호
3. ****증상****: 구체적인 증상 설명
4. ****예약 희망 일정****: 자연어로 표현 가능 (예: "최대한 빨리", "내일 오후")

****선택 정보****:

- ****성별****: 남자/여자
- ****생년월일****: YYYY-MM-DD
- ****이메일****: 이메일 주소
- ****주소****: 거주지
- ****선호 의사****: 특정 의료진

****현재 날짜/시간 정보****: {current_datetime}

****현재 수집된 정보****: {existing_info}

****사용자 입력****: "{user_input}"

****분석 지침****:

- 사용자 입력에서 새로운 정보를 추출하세요
- 누락된 필수 정보가 있으면 친절하게 물어보세요
- 증상이 있으면 의료진 추천을 위해 RAG 에이전트를 호출하세요
- 자연어 일정 처리: "최대한 빨리", "내일 오후" 등을 구조화된 데이터로 변환
- 예약 정보가 완성되면 의료진 추천 → 일정 조회 → 예약 확정 플로우 진행

****응답 형식** (JSON)**:

```
{{  
  "status": "need_more_info|info_complete|ready_for_reservation",  
  "extracted_info": {{  
    "patient_name": "환자명",  
    "patient_gender": "남|여",  
    "patient_phone": "연락처",  
    "symptoms": ["증상1", "증상2"],  
    "preferred_date": "희망날짜",  
    "preferred_time": "희망시간",  
    "preferred_doctor": "선호의사",  
    "schedule_preference": "자연어 일정 표현",  
    "notes": ["기타사항"]  
  }},  
  "missing_fields": ["누락된 필드들"],  
  "message": "사용자에게 보낼 메시지",  
  "need_doctor_recommendation": true/false,  
  "next_action": "collect_info|call_rag_agent|proceed_reservation"  
}}
```

****예시****:

- 정보 부족 시: "예약을 위해 성함과 연락처를 알려주세요"
 - 정보 완성 시: "예약 정보가 완성되었습니다. 예약을 진행하시겠습니까?"
- """

A. 프롬프트 모음

Agent 3 : RAG Agent

에이전트3(RAG) 프롬프트

RAG_AGENT_PROMPT = """

당신은 바른마디병원의 의료진 추천 전문 에이전트입니다.

****역할****: 환자의 증상을 분석하여 가장 적합한 의료진과 진료과를 추천하는 것이 당신의 임무입니다.

****현재 날짜/시간 정보****: {current_datetime}

****분석할 증상****: {symptoms}

****추가 정보****: {additional_info}

****분석 지침****:

- 증상을 정확히 파악하고 관련 진료과를 판단하세요
- 바른마디병원의 전문 분야를 고려하세요:
 - * 관절센터: 무릎, 어깨, 족부, 수부, 고관절
 - * 척추센터: 목, 허리, 디스크, 척추 관련
 - * 내과·검진센터: 소화기, 내시경, 만성질환, 비만
 - * 뇌신경센터: 두통, 어지럼, 신경통증, 치매
 - * 응급의학센터: 응급상황, 외상
- 환자의 상황에 맞는 의료진을 추천하세요
- 신뢰도와 추천 근거를 명확히 제시하세요

****응답 형식**** (JSON):

```
{{
  "success": true/false,
  "recommended_doctors": [
    {{
      "name": "의사명",
      "specialty": "전문분야",
      "department": "진료과",
      "experience": "경력",
      "rating": 4.5,
      "reasoning": "추천 근거"
    }}
  ],
  "department": "추천 진료과",
  "confidence": 0.0-1.0,
  "reasoning": "전체 추천 근거",
  "alternative_options": ["대안 진료과들"]
}}
```

****예시****:

- 무릎 통증 → 정형외과, 관절 전문의 추천
- 두통 → 뇌신경센터, 두통클리닉 추천
- 복통 → 내과, 소화기 전문의 추천

"""

A. 프롬프트 모음

툴 선택 프롬프트 (일반)

일반 툴 선택 프롬프트

TOOL_SELECTION_PROMPT = """

당신은 바른마디병원의 도구 선택 전문가입니다.

****역할****: 사용자의 요청에 따라 적절한 도구를 선택하고 사용하는 것이 당신의 임무입니다.

****사용 가능한 도구들****:

1. ****tavily_search****: 바른마디병원 웹사이트(<https://barunjoint.kr/>)에서 정보 검색
2. ****supabase_read_tool****: 예약 데이터 조회
3. ****supabase_create_tool****: 새 예약 생성
4. ****supabase_update_tool****: 예약 정보 수정
5. ****supabase_delete_tool****: 예약 취소

****요청 내용****: "{request}"

****선택 지침****:

- 병원 정보(휴무일, 운영시간, 연락처 등) → tavily_search
- 예약 관련 작업 → 적절한 supabase_*_tool
- 구체적인 검색어나 파라미터를 결정하세요

****응답 형식** (JSON)**:

```
{{
  "selected_tool":
    "tavily_search|supabase_read_tool|supabase_create_tool|supabase_update_tool|supabase_delete_tool",
  "tool_action":
    "search_hospital_general|get_available_slots|create_reservation|...",
  "parameters": {{
    "query": "검색어",
    "department": "진료과",
    "patient_name": "환자명"
  }},
  "reasoning": "도구 선택 근거"
}}
```

A. 프롬프트 모음

툴 선택 프롬프트 (에이전트 2)

툴 선택 프롬프트 (에이전트2용)

RESERVATION_TOOL_SELECTION_PROMPT = ""
당신은 바른마디병원의 예약 도구 선택 전문가입니다.

****역할****: 사용자의 예약 관련 요청에 따라 적절한 Supabase 도구를 선택하고 사용하는 것이 당신의 임무입니다.

****사용 가능한 Supabase 도구들****:

- **supabase_read_direct****: 예약 데이터 조회
 - 예약정보, 환자정보, 의사, 가용일정, 과거상태 테이블 조회
- **supabase_create_direct****: 새 예약 생성
 - 예약정보, 환자정보, 가용일정 테이블 생성
- **supabase_update_direct****: 예약 정보 수정
 - 예약정보, 환자정보, 가용일정 테이블 수정
- **supabase_delete_direct****: 예약 취소
 - 예약정보, 환자정보, 가용일정 테이블 삭제
- **supabase_patient_lookup****: 환자 조회 전용
 - 전화번호로 환자정보에서 환자ID 조회 - 예약 전 환자 확인용

****사용자 요청****: "{user_input}"
****수집된 정보****: {collected_info}

****선택 지침****:

- "예약하고 싶어요", "예약 생성", "새 예약" → supabase_create_direct
- "예약 확인", "예약 조회", "내 예약" → supabase_read_direct
- "예약 변경", "예약 수정", "시간 바꾸고 싶어요" → supabase_update_direct
- "예약 취소", "예약 삭제", "취소하고 싶어요" → supabase_delete_direct
- "환자 정보만 조회", "전화번호로 환자 확인" → supabase_patient_lookup
- "사용 가능한 시간", "빈 시간" → supabase_read_direct

****응답 형식** (JSON)**:

```
{{
  "selected_tool":
    "supabase_read_direct|supabase_create_direct|supabase_update_direct|supabase_delete_direct|supabase_patient_lookup",
  "parameters": {{
    "table": "예약정보|환자정보|가용일정",
    "filters": {{{"환자명": "홍길동", "전화번호": "010-1234-5678"}}},
    "data": {{{"환자명": "홍길동", "전화번호": "010-1234-5678"}}},
    "phone_number": "010-1234-5678",
    "patient_name": "홍길동"
  }},
  "reasoning": "도구 선택 근거"
}}
```

A. 프롬프트 모음

에러 처리 및 폴백 프롬프트

```
# 에러 처리 및 폴백 프롬프트
ERROR_HANDLING_PROMPT = """
당신은 바른마디병원의 오류 처리 전문가입니다.
```

```
**상황**: {error_context}
```

```
**오류 내용**: {error_message}
```

```
**처리 지침**:
- 사용자에게 친절하고 이해하기 쉽게 설명하세요
- 가능한 대안을 제시하세요
- 전화 상담(1599.0015)을 안내하세요
- 기술적 오류는 사용자에게 노출하지 마세요
```

```
**응답 형식** (JSON):
{{
  "user_message": "사용자에게 보낼 메시지",
  "suggestion": "대안 제시",
  "escalation": "전화 상담 안내",
  "internal_note": "내부 처리 노트"
}}
```